

OrgElMorph

An Onboarding Course in Organic-Electronics Morphology Simulation
Physics · Computation · Differentiation

DiffSim project

Abstract

This course teaches the simulation of *morphology* in organic-electronic thin films — the coupled phase separation and crystallization of polymer/small-molecule/solvent blends that decides how organic solar cells, transistors, and bioelectronic devices perform. It is built around **OrgElMorph**, a GPU-native, differentiable phase-field simulator. Three tracks run in parallel: the *physics* of the models, the *computation* that makes them fast and correct, and the *differentiation* that turns the simulator into an instrument for design and inference. Every concept pairs a written explanation with a runnable tutorial; you reproduce the figures and numbers here, then explore the questions on your own.

Contents

1	Using this course	5
I	Physics	6
2	Binary Cahn–Hilliard: two free energies	7
2.1	The model	7
2.2	Two bulk free energies	8
2.3	Run it	8
2.4	Code walk-through	10
2.5	Explore on your own	11
3	Adaptive time stepping	12
3.1	The model	12
3.2	Coefficients	13
3.3	Run it	13
3.4	Code walk-through	14
3.5	Explore on your own	14
4	Substrate and boundary conditions	16
4.1	The model	16
4.2	Coefficients	16
4.3	Run it	17
4.4	Code walk-through	18
4.5	Explore on your own	19
5	Ternary Cahn–Hilliard and the phase diagram	20
5.1	The model	20
5.2	The Gibbs triangle	20
5.3	Run it	21
5.4	Code walk-through	22
5.5	Explore on your own	22
6	Evaporation: the drying film	23
6.1	The model	23
6.2	Run it	23
6.3	Code walk-through	25
6.4	Explore on your own	25

7	Allen–Cahn crystallization	26
7.1	The model	26
7.2	Growth kinetics: the Avrami law	26
7.3	Run it	27
7.4	Code walk-through	28
7.5	Explore on your own	28
8	Coupled Cahn–Hilliard + Allen–Cahn	29
8.1	The model: the four-fold χ	29
8.2	Run it	29
8.3	Code walk-through	31
8.4	Explore on your own	31
9	Thermal noise and nucleation	32
9.1	The model	32
9.2	Run it	32
9.3	Code walk-through	34
9.4	Explore on your own	34
10	Evaporation-induced crystallization	35
10.1	The model	35
10.2	Run it	35
10.3	Code walk-through	37
10.4	Explore on your own	37
II	Computation	39
11	Convergence: basis order and time integration	40
11.1	The two kinds of error	40
11.2	Spatial order: the method of manufactured solutions	40
11.3	Temporal order: self-convergence	41
11.4	Run it	41
11.5	Code walk-through	42
11.6	Explore on your own	43
12	Boundary conditions, numerically	45
12.1	Boundaries live in the weak form	45
12.2	Run it	46
12.3	Code walk-through	47
12.4	Explore on your own	48
13	Spatial and temporal adaptivity	49
13.1	Spatial adaptivity: octree refinement	49
13.2	Temporal adaptivity: the LTE step ladder	49
13.3	Why variable-coefficient BDF2	50
13.4	Code walk-through	52
13.5	Explore on your own	52

14 The solver ecosystem	53
14.1 Three families of solver	53
14.2 The 2-D crossover, measured live	54
14.3 The 3-D story, measured (and cited)	54
14.4 Code walk-through	55
14.5 Explore on your own	56
15 Two dimensions versus three	57
15.1 Three things grow at once	57
15.2 Run it	57
15.3 The device-scale ladder	58
15.4 Code walk-through	59
15.5 Explore on your own	60
III Differentiation	61
16 What differentiable simulation is	62
16.1 Why a gradient	62
16.2 The three ways to a gradient	62
16.3 Run it	63
16.4 Code walk-through	63
16.5 Explore on your own	64
17 Morphology sensitivity	66
17.1 A scalar handle on the morphology	66
17.2 Run it	66
17.3 Code walk-through	67
17.4 Explore on your own	67
18 Recovering parameters from a morphology	69
18.1 The inverse problem	69
18.2 Run it	69
18.3 Code walk-through	70
18.4 Explore on your own	71
19 Process gradients	72
19.1 Temperature enters everywhere	72
19.2 Run it	72
19.3 Code walk-through	73
19.4 Explore on your own	74
20 Crystallinity sensitivity	75
20.1 Differentiating the crystalline order	75
20.2 Run it	75
20.3 A frontier, named honestly	76
20.4 Explore on your own	76

21 Learning the free energy from snapshots	78
21.1 A learnable free energy	78
21.2 Run it	78
21.3 Code walk-through	79
21.4 Explore on your own	80
22 Capstone: inverse-designing a process	81
22.1 Design as optimisation	81
22.2 Run it	81
22.3 Code walk-through	82
22.4 Explore on your own	83

Chapter 1

Using this course

Read a concept below, then open its tutorial folder (`tutorials/orgelmorph-course/<track>/<NN_name>/`), run `run.py`, and compare your printed self-check table and figures with the ones in this document. The tutorials call the same production solver the research code uses. The three tracks cross-reference each other; a first pass straight down the Physics track is a good start, dipping into the matching Computational and Differentiable concepts as they are referenced.

Boxes flag the two things you act on:

Key idea. The load-bearing concept.

Run it. The command, and what to compare.

Questions marked **Q** are for you to explore — they have no answers printed here on purpose.

Part I

Physics

Chapter 2

Binary Cahn–Hilliard: two free energies

Key idea. A blend of two components lowers its free energy by *demixing*. The Cahn–Hilliard equation is the conserved gradient flow of a Ginzburg–Landau free energy; that free energy can only decrease in time, and it splits into a *bulk* (mixing) part and an *interfacial* (gradient) part whose competition sets the morphology and its length scale.

2.1 The model

Let $c(\mathbf{x}, t)$ be the local volume fraction of one component (a *conserved* order parameter: the total amount of each species is fixed). The Ginzburg–Landau free energy is

$$F[c] = \int_{\Omega} \left[f(c) + \frac{\kappa}{2} |\nabla c|^2 \right] dV, \quad (2.1)$$

with two contributions:

- the **bulk** (homogeneous) free energy density $f(c)$, which is minimized by the pure phases and *penalized* in between — this is what drives demixing;
- the **interfacial** term $\frac{\kappa}{2} |\nabla c|^2$, which penalizes spatial variation of c and therefore *opposes* demixing at short scales. The coefficient $\kappa > 0$ has units of energy $\times$ length² and sets the interface width $\ell \sim \sqrt{\kappa}$.

Because c is conserved, the dynamics is the *conserved* (Model-B) gradient flow of (2.1): mass moves down gradients of the chemical potential $\mu = \delta F / \delta c$,

$$\frac{\partial c}{\partial t} = \nabla \cdot (M \nabla \mu), \quad \mu = f'(c) - \kappa \nabla^2 c. \quad (2.2)$$

Here $M > 0$ is the mobility (how fast mass responds to a chemical-potential gradient; it sets the *time* scale but not the equilibrium morphology). The fourth-order PDE is split into the two second-order equations (2.2) — the *mixed* (c, μ) form the solver uses.

F is a Lyapunov functional. Taking dF/dt along (2.2) and integrating by parts with no-flux boundaries gives $dF/dt = -\int_{\Omega} M |\nabla \mu|^2 dV \leq 0$. The free energy can only decrease.

Watching it decrease — and split between bulk and interface — is how you *see* the physics in the numbers.

2.2 Two bulk free energies

Polynomial double well (the canonical model). The simplest f with two equal minima,

$$f_{\text{poly}}(c) = \frac{1}{4}(c^2 - 1)^2, \quad f' = c^3 - c, \quad f'' = 3c^2 - 1, \quad c \in [-1, 1]. \quad (2.3)$$

The two phases are $c = \pm 1$; the region $f'' < 0$ (i.e. $|c| < 1/\sqrt{3}$) is *spinodally unstable* — any small fluctuation there grows. This model is dimensionless and symmetric; it isolates the phase-field mechanism from material detail.

Flory–Huggins (the physical model). For a real blend the mixing free energy is entropic + enthalpic,

$$f_{\text{fh}}(c) = A [c \ln c + (1 - c) \ln(1 - c)] + B c(1 - c), \quad c \in (0, 1), \quad (2.4)$$

where the logarithmic *entropy of mixing* (scale $A \sim 1/N$, with N the degree of polymerization) always favors mixing, and the *enthalpic* term $B c(1 - c)$ (with B the Flory interaction parameter χ) favors demixing when the components dislike each other. The curvature

$$f''_{\text{fh}}(c) = A \left(\frac{1}{c} + \frac{1}{1-c} \right) - 2B \quad (2.5)$$

is negative — unstable — only where the enthalpic $-2B$ beats the (always convex) entropic term. This is the *spinodal*. The logs diverge at the pure phases, so Flory–Huggins keeps c strictly inside $(0, 1)$ (no unphysical overshoot), at the cost of a stiff f'' near the walls — a numerical point we return to in the Computational track.

Coefficients and their ranges. $M > 0$ (mobility, time scale). $\kappa > 0$ (gradient penalty; interface width $\sqrt{\kappa}$ — choose it so the interface spans ~ 3 – 5 mesh cells). Polynomial: no free parameters. Flory–Huggins: $A > 0$ (entropy, $\sim 1/N$; $A \rightarrow 0$ for long polymers), $B = \chi$ (interaction; demixing requires B large enough that $f'' < 0$ somewhere — for $A = 1$ the symmetric point $c = \frac{1}{2}$ is unstable once $B > 2A = 2$). The tutorial uses $A = 1$, $B = 2.5$ (a clear quench); real blends map $A \sim 1/N$, $B \sim \chi$ (see `materials/materials.yaml`).

2.3 Run it

Run it.

```
cd physics/01_ch_binary_energies
python run.py # both free energies, 128x128
```

It marches a spinodal decomposition from a nearly uniform blend ($c = \bar{c}$ + small noise) for both free energies, printing the energy budget. Compare the printed table with `EXPECTED.md` and the figures below.

What you should see. From a nearly uniform, slightly noisy blend, small fluctuations in the unstable region grow (spinodal decomposition), sharpen into domains separated by thin interfaces, and then *coarsen*. The total free energy drops monotonically (after a one-step start-up transient discussed below), mass is conserved to solver tolerance, and the field saturates at the two coexisting phases.

On the random seed. The initial perturbation is drawn from a *fixed* random seed (set in `spinodal.py`), so every run is bit-reproducible and matches the figures here — that is what lets you *compare* against the document. Note the distinction: the *statistics* (energy decay, mass conservation, the phase values $c_{\min}, c_{\max}$) are seed-independent physics, but the *exact morphology* — which domain forms where — is not. Change the seed and the pattern changes while the energy curve does not. (Later, in the Computational and Physics-noise concepts, the seed becomes a first-class object: convergence must be seed-robust, and thermal *nucleation* is seeded by physical fluctuations.)

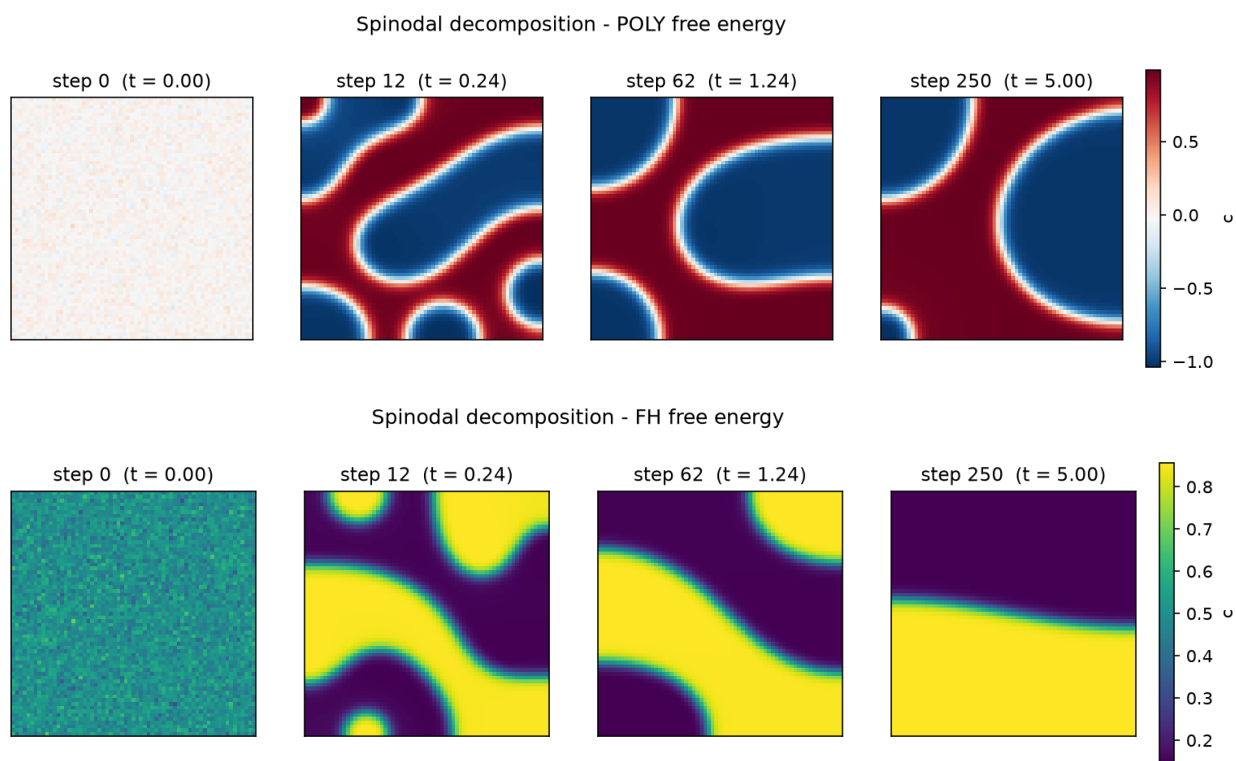


Figure 2.1: Spinodal decomposition and coarsening for the polynomial (top, $c \in [-1, 1]$) and Flory–Huggins (bottom, $c \in (0, 1)$) free energies, at the mesh and parameters of the tutorial. Same mixing physics, different order-parameter range and interface structure.

Measured self-check (your `run.py` should reproduce these to a few significant figures; exact values depend on the RNG seed fixed in the tutorial).

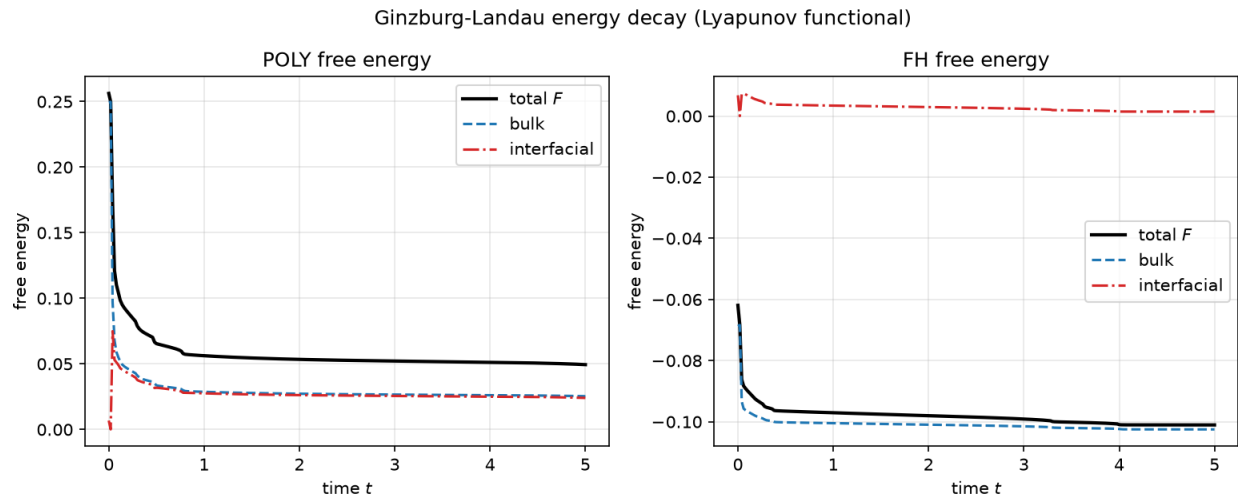


Figure 2.2: The Ginzburg–Landau energy budget: total free energy (black) and its bulk (blue) and interfacial (red) parts versus time. The total is a Lyapunov functional — monotone decreasing. Early on, forming interfaces *raises* the interfacial energy while the bulk energy falls faster; during coarsening, domains merge and the interfacial energy falls as total interface area shrinks.

	polynomial	Flory–Huggins
total F : start $\rightarrow$ end	0.256 $\rightarrow$ 0.0494	-0.0619 $\rightarrow$ -0.101
interfacial energy: peak $\rightarrow$ end	0.0749 $\rightarrow$ 0.024	0.00741 $\rightarrow$ 0.00146
phase values $[c_{\min}, c_{\max}]$	$[-1.04, 0.99]$	$[0.14, 0.86]$
mass drift $ \Delta m $	4.1×10^{-16}	$< 10^{-14}$ (machine)
F monotone after step 1	yes	yes

2.4 Code walk-through

The tutorial’s physics lives in `spinodal.py`, which drives the production brick `src/diffsim/physics/cahn_hilliard.py`. The three sections you interact with:

1. Build a mesh and the stepper.

```
dm, mesh, cons = build_mesh_dm(level=7, p=1, device="cuda:0") # 128x128
st = CahnHilliardStepper(dm, M, kappa, dt, order=1,
                        energy="poly") # or energy="fh"
```

energy selects $f(c)$; order is the time scheme (1 = backward Euler); M , $kappa$ are the coefficients above. The constructor is where the *model* is chosen — everything else is running it.

2. Seed the initial blend.

```
st.set_initial(lambda x: c_avg + amp*rng.standard_normal(len(x)),
              mu_init="consistent")
```

A nearly uniform field with a small random perturbation — the seed of the spinodal instability. `mu_init="consistent"` projects a matching initial chemical potential so the very first step does not see an artificial μ -transient (the one-step start-up you see in Fig. 2.2).

3. March, and measure the energy each step.

```
for n in range(steps):  
    c, mu = st.step()  
    F_total, F_bulk, F_int = energy_budget(st, dm, mesh, c, ...)
```

`step()` advances one time step (assemble $\rightarrow$ Newton solve on the GPU). `energy_budget` evaluates c and ∇c at the Gauss points — using the same quadrature the solver uses — and integrates the two energy densities of (2.1) separately. That is the whole measurement: no post-processing magic, just the definition of F .

2.5 Explore on your own

Q2.5.1 Vary κ over, say, $\{2, 5, 10\} \times 10^{-4}$. How does the equilibrium interface width scale, and how many mesh cells does the interface span in each case? At what point do you stop trusting the result, and why?

Q2.5.2 For Flory–Huggins, sweep B (the interaction χ) from below to above the spinodal threshold $2A$. Where does demixing stop happening, and does the onset match $f''(\frac{1}{2}) = 0$?

Q2.5.3 The polynomial model is symmetric ($\bar{c} = 0$); Flory–Huggins with $\bar{c} \neq \frac{1}{2}$ is not. Run an off-critical blend ($\bar{c} = 0.35$) and describe how the morphology differs from the symmetric case (droplets vs. bicontinuous). Which blend ratio gives which?

Q2.5.4 The total energy dips monotonically but the interfacial energy is *non-monotone*. Explain the two regimes (interface formation vs. coarsening) in terms of F_{bulk} and F_{int} , and predict how each scales during coarsening.

Q2.5.5 Mobility M was said to set the time scale but not the equilibrium. Test it: double M and check whether the *final* morphology and energy are unchanged while the *approach* rescales in time.

Chapter 3

Adaptive time stepping

Key idea. Coarsening is multi-scale *in time*: the spinodal quench is violent (interfaces form in a few very small time steps) and then coarsening slows down geometrically (domains merge ever more rarely). A single fixed time step cannot serve both regimes. An *error-controlled* step—small during the quench, large through the tail—resolves the physics the fixed step misses, at a fraction of the cost.

3.1 The model

The equation is unchanged from Chapter 2: the conserved gradient flow (2.2) of the Ginzburg–Landau energy (2.1). What changes is the time integration. Writing the time derivative with a backward-difference formula (BDF) of order p ,

$$\left. \frac{\partial c}{\partial t} \right|_{t_{n+1}} \approx \frac{1}{\Delta t} \left(\sigma c^{n+1} - \text{hist}(c^n, c^{n-1}) \right), \quad (3.1)$$

turns each step into a nonlinear (Newton) solve. The question is how to *choose* Δt .

Local truncation error control (step doubling). From a state c^n , take one step of size Δt to get $c_{\Delta t}$, and *separately* two steps of $\Delta t/2$ to get $c_{\Delta t/2}$. The two differ by the local truncation error (LTE),

$$\text{LTE} \approx \frac{\|c_{\Delta t} - c_{\Delta t/2}\|_2}{2^p - 1}. \quad (3.2)$$

Accept the (more accurate) half-step solution if $\text{LTE} < \text{tol}$, and propose the next step from the standard controller

$$\Delta t_{\text{new}} = \Delta t \cdot \text{clamp}\left(\text{safety}(\text{tol}/\text{LTE})^{1/(p+1)}, \frac{1}{2}, 2\right); \quad (3.3)$$

otherwise *reject*, halve Δt , and retry. This is the classic Wodo–Ganapathysubramanian phase-field controller (JCP 230 (2011) 6037). The time step *grows* through coarsening—the property that makes long phase-field horizons affordable.

Why the BDF2 must be variable-coefficient. A constant-coefficient BDF2 is only second-order when the step is *constant*. Under an adaptive Δt the step ratio $r = \Delta t_n / \Delta t_{n-1}$ varies, so the stepper rebuilds the BDF2 weights from the actual $(\Delta t_n, \Delta t_{n-1})$ every step; $r = 1$ reproduces the fixed-step scheme bit-exactly. The order verification lives in the Computational

track (temporal adaptivity).

3.2 Coefficients

Controller knobs and their meaning. *tol* (the accuracy target on the relative LTE; smaller $\Rightarrow$ smaller steps and a more accurate trajectory). *p* the BDF order (here 2). *safety* $\in (0, 1)$ the fraction of the predicted optimal step actually taken (robustness margin; 0.85 is typical). $\Delta t_{\min}, \Delta t_{\max}$ hard bounds. The *physics* coefficients (M, κ , the free energy) are exactly those of Chapter 2: adaptivity changes cost and accuracy, not the model.

3.3 Run it

Run it.

```
cd physics/02_ch_adaptive_stepping
python run.py                # naive fixed vs adaptive, 64x64
```

It marches the same binary quench to $t = 0.8$ with a naive fixed step ($\Delta t = 0.004$) and with the adaptive controller ($\text{tol} = 0.002$), and prints the cost/accuracy comparison. Compare with `EXPECTED.md` and the figures below.

What you should see. The adaptive step size spans several orders of magnitude—collapsing to $\Delta t \approx 1.0 \times 10^{-6}$ through the quench (where the truncation error is largest) and climbing to $\Delta t \approx 0.050$ through coarsening. The naive fixed step (200 steps) is *both* more expensive than the adaptive run (97 steps, $2.1\times$ fewer) *and* less accurate: its energy is stuck at 0.1156, well above the adaptive run’s more-relaxed 0.08047, because it under-resolves the quench. A fixed step *safe* for the whole quench (i.e. $\Delta t \leq 1.0 \times 10^{-6}$) would need on the order of 800 000 steps—the cost the controller avoids.

The morphology is step-sequence dependent—like the seed. Two integrators that take different step sequences produce different *coarsened microstructures* (which domain merges when is chaotically sensitive), exactly as changing the RNG seed does in Chapter 2. What is *robust* is the physics: the free energy, the characteristic domain scale, and the phase values. So we compare those statistics, not the pixelwise field.

Measured self-check (your `run.py` should reproduce these to a few significant figures).

	naive fixed	adaptive
time steps taken	200	97
final free energy F	0.1156	0.08047
Δt range	0.004 (held)	$1.0 \times 10^{-6} \rightarrow 0.050$
domain scale (cells)	25.8	25.1
phase values $[c_{\min}, c_{\max}]$	[−1.00, 1.00]	

Step-count saving $2.1\times$; energy difference $|\Delta F| = 3.5 \times 10^{-2}$ (the fixed step lags, not a match: it is less accurate).

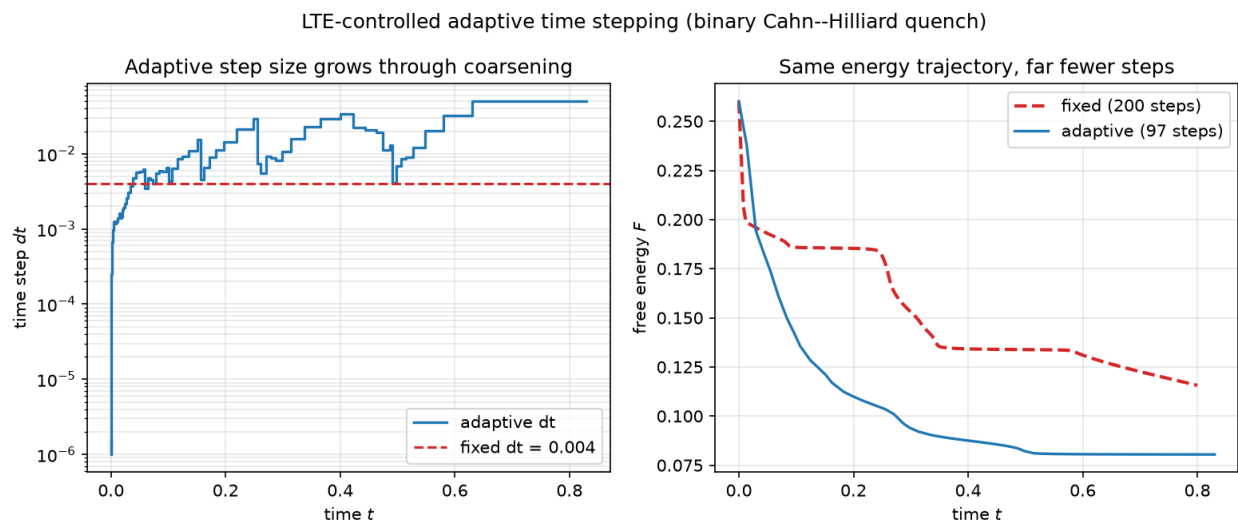


Figure 3.1: Left: the adaptive time step over the quench—tiny through the spinodal onset, growing by orders of magnitude through coarsening (the fixed step is the dashed line). Right: the free-energy history; the adaptive run (fewer steps) reaches a *lower* (more relaxed) energy than the naive fixed step, which lags because it under-resolves the quench.

3.4 Code walk-through

1. The adaptive march. The controller is a library function; the tutorial calls it in short checkpoint windows so it can record the energy and step size along the way:

```
from diffsim.physics.cahn_hilliard import adaptive_march
ts, dts = adaptive_march(stepper, t_end, tol=2e-3,
                          dt_max=0.05, dt_min=1e-6)
```

`adaptive_march` owns the accept/reject ladder above; it returns the accepted $(t, \Delta t)$ pairs, from which the `gen_figures.py` builds the Δt history.

2. The variable-step BDF2. Inside `stepper.step()` the BDF2 weights are recomputed every step from $(\Delta t_n, \Delta t_{n-1})$, so a rejected-and-halved step stays second-order consistent—you do not have to do anything: the accuracy the controller assumes is actually delivered.

3. The cost/accuracy readout. `compare()` runs both integrators to the same horizon and returns the step counts, the energy histories, and the robust statistics (domain scale, phase values) that *should* agree even though the fields do not.

3.5 Explore on your own

Q3.5.1 Tighten the tolerance (`--tol 5e-4`). How do the number of steps, the final energy, and $\Delta t_{\max}$ change? Does the final energy *converge* as `tol` shrinks (evidence the adaptive result is accurate), and roughly at what `tol` does it stop moving?

Q3.5.2 The adaptive run needed $\Delta t \sim 1.0 \times 10^{-6}$ during the quench. Estimate the largest fixed Δt that reproduces the adaptive final energy, and compare its step count to the adaptive count.

Coarsened morphology: the exact pattern is step-sequence dependent (as with the RNG seed),
but the statistics (energy, domain scale, phases) agree

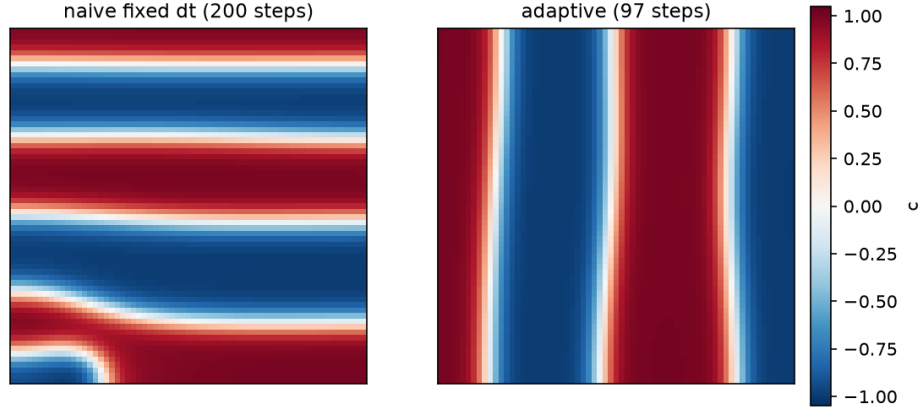


Figure 3.2: Final coarsened morphology from the two integrators. The exact pattern differs (step-sequence sensitivity, as with the RNG seed), while the domain scale and phase values agree.

What is the true speed-up at matched accuracy?

Q3.5.3 Each adaptive step costs one full plus two half solves (step doubling). Given that, at what point in the coarsening does adaptivity start *paying for itself*, and why is it a net loss during the quench?

Q3.5.4 Change the RNG seed and rerun both integrators. Which of the reported quantities move (morphology, domain scale, energy, phase values) and which do not? Relate this to the seed discussion in Chapter 2.

Q3.5.5 Increase κ (wider interfaces). Does the quench become *less* stiff (larger $\Delta t_{\min}$)? Explain the link between the interface width and the smallest time step the controller must take.

Chapter 4

Substrate and boundary conditions

Key idea. A cast film is bounded by a *substrate* and by *air*, and those interfaces are usually not neutral: one component prefers the wall. That preference is imposed through the *boundary condition*, and it sets the *vertical stratification* of the morphology—which in a device decides whether the right material meets the right electrode.

4.1 The model

The bulk equations are binary Cahn–Hilliard (Chapter 2); the new physics lives on the boundary. We keep the natural no-flux condition on the sides and add a *substrate wall free energy* on the bottom edge Γ_w ,

$$F_w = \int_{\Gamma_w} f_w(\phi) dS, \quad f_w(\phi) = g\phi + h\phi^2, \quad (4.1)$$

a surface energy density that is minimized at the preferred surface composition $\phi^* = -g/(2h)$. Taking the variation of the total free energy adds a *natural boundary term* to the chemical-potential equation—the wall condition is

$$\kappa \frac{\partial \phi}{\partial n} \Big|_{\Gamma_w} = -f'_w(\phi) = -(g + 2h\phi), \quad (4.2)$$

so the μ residual gains $-\int_{\Gamma_w} N_a (g + 2h\phi) dS$ (the consistent, basis-generic face term). Crucially this is a condition on ϕ , not on the *mass* flux: the flux boundary condition $\nabla \mu \cdot n = 0$ is unchanged, so **mass is still conserved**—the wall changes *where* material wants to be, not how much there is.

Boundary-condition taxonomy. A phase-field boundary can carry: (i) *natural / no-flux* ($\nabla \mu \cdot n = \nabla \phi \cdot n = 0$): neutral, nothing crosses, mass conserved—the default; (ii) *wall energy* (this chapter): a Neumann-type condition on $\nabla \phi$ from the surface energy f'_w ; still no mass flux, but the equilibrium is stratified; (iii) *Dirichlet*: pin ϕ or μ to a prescribed value (a reservoir / fixed-composition contact)—this *does* exchange mass and is used mainly for manufactured-solution verification (Computational track).

4.2 Coefficients

Wall coefficients and their meaning. g (linear surface field; $g < 0$ *attracts* the component, $g > 0$ repels it), $h > 0$ (curvature of the well, bounding the preference). Their ratio sets the preferred surface composition $\phi^* = -g/(2h)$, which must lie strictly inside $(0, 1)$ (a preference outside the physical range would drive ϕ into the wall, where the Flory–Huggins logarithm is stiff). The tutorial uses a weakly demixing blend ($\chi = 2.2$) with $\phi^* = 0.75$ (attracting) and $\phi^* = 0.25$ (repelling) around a $\bar{\phi} = 0.5$ blend. Real substrate energies are measured as contact-angle / surface- energy asymmetries.

4.3 Run it

Run it.

```
cd physics/03_ch_substrate_bcs
python run.py                # neutral vs attracting vs repelling wall
```

It marches the same blend under three substrate conditions and prints the substrate composition, the film-mean composition, the enrichment, and the mass drift. Compare with `EXPECTED.md` and the figures.

What you should see. The neutral wall leaves the composition unstratified (substrate ≈ 0.493 , enrichment -0.007). The attracting wall pulls the component to the substrate (substrate ≈ 0.750 , enrichment $+0.248$); the repelling wall depletes it (substrate ≈ 0.250 , enrichment -0.249). In every case the mass drift is $\leq 2.0 \times 10^{-3}$: the wall re-arranges material without creating or destroying it.

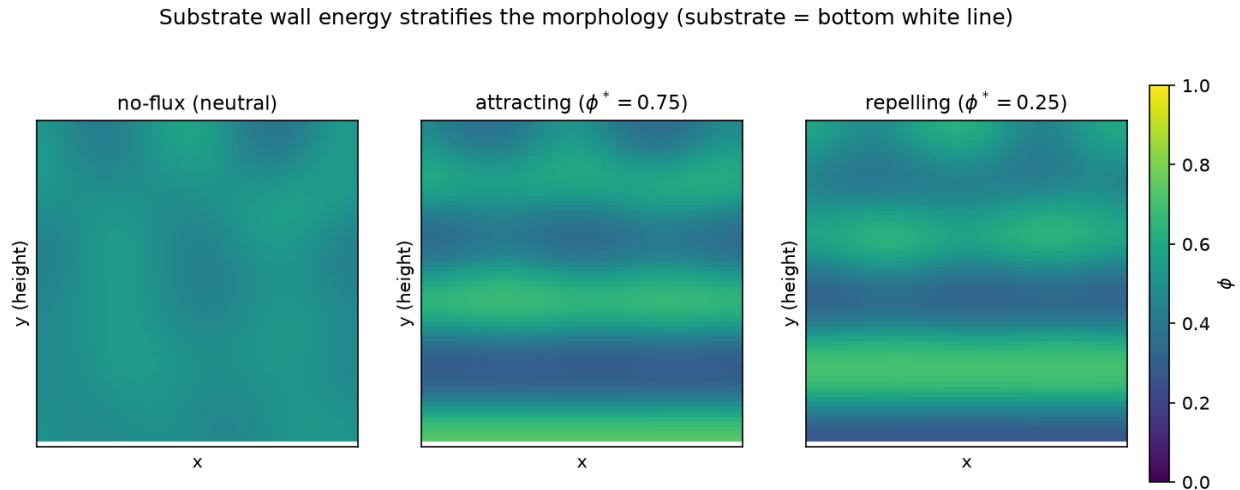


Figure 4.1: The composition field under the three substrate conditions (substrate = the bottom edge). The attracting wall enriches the bottom layer; the repelling wall depletes it; the neutral wall does neither.

Measured self-check (your `run.py` should reproduce these).

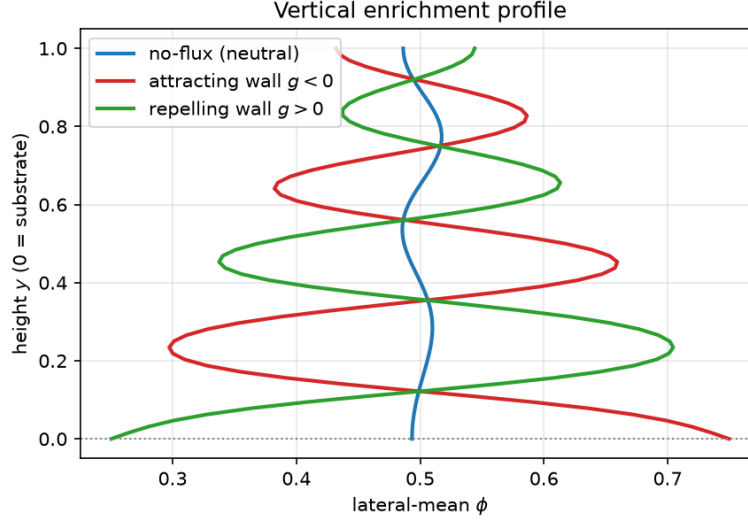


Figure 4.2: Vertical enrichment profile (lateral-mean ϕ vs height). The attracting/repelling walls bend the profile up/down at the substrate; the neutral profile is flat. This boundary layer is the substrate’s fingerprint on the morphology.

	neutral	attracting	repelling
substrate ϕ	0.493	0.750	0.250
enrichment $\Delta\phi$	-0.007	+0.248	-0.249

Mass drift $\leq 2.0 \times 10^{-3}$ in all cases (conservation survives the wall energy).

4.4 Code walk-through

1. Turn on the wall. The wall energy is three constructor arguments on the production brick:

```
st = MultiPhaseStepper(dm, M=1, K=0, chi_aa=..., onsager=[[0.2]],
                        wall_g=[-1.5], wall_h=[1.0], wall_face=(1, 0))
```

`wall_face=(1,0)` selects the $y = 0$ (axis 1, min side) substrate edge; `wall_g`, `wall_h` are g, h of (4.1). Setting both to zero recovers the neutral no-flux boundary bit-for-bit.

2. Read the enrichment. The tutorial builds a robust node→grid map (so “substrate row” is exactly the y -minimum) and reports the lateral-mean ϕ per height—the enrichment profile of Fig. 4.2.

3. Conditioning. A low Onsager mobility (0.2) and the *bounded* quadratic well keep Newton at ~ 4 iterations per step; a preferred composition outside $(0, 1)$, or a much larger mobility, drives ϕ into the Flory–Huggins wall and collapses the step size (a numerical point revisited in the Computational track).

4.5 Explore on your own

Q4.5.1 Move the preferred composition ϕ^* toward 0 or 1 (increase $|g|$ at fixed h). At what point does the substrate layer hit the Flory–Huggins wall, and how does the time step respond? Relate this to the stiffness discussion in Chapter 3.

Q4.5.2 Put the wall on the *air* side (`wall_face=(1,1)`) instead of the substrate, with the opposite preference. Does the morphology invert? What does this imply for which component ends up at the top vs bottom electrode?

Q4.5.3 Raise χ above the spinodal so the blend also demixes. Does the attracting wall bias *which* phase wets the substrate? Describe the coupling between bulk phase separation and the surface preference.

Q4.5.4 Verify conservation explicitly: track the total mass over the run for each boundary condition. Why does the wall energy, unlike a Dirichlet condition, leave the mass exactly fixed?

Q4.5.5 Estimate the thickness of the enrichment boundary layer and relate it to $\sqrt{\kappa}$ and to $\phi^* - \bar{\phi}$. How would you change κ to make the substrate layer thinner?

Chapter 5

Ternary Cahn–Hilliard and the phase diagram

Key idea. A real cast blend has (at least) three components — two solutes and a solvent. With two independent conserved compositions the free energy is a *surface over the Gibbs triangle*, and demixing follows *tie-lines* on that triangle. Reading morphology on the phase diagram, not on a single axis, is the multi-component way of thinking.

5.1 The model

Let ϕ_1, ϕ_2 be the volume fractions of the two solutes; the solvent is eliminated by $\phi_s = 1 - \phi_1 - \phi_2$. Each solute carries a conserved (ϕ_i, μ_i) Cahn–Hilliard pair; the Flory–Huggins exchange chemical potentials are

$$\mu_i^{\text{bulk}} = \ln \frac{\phi_i}{\phi_s} + \chi_{12} \phi_j + \chi_{is}(\phi_s - \phi_i) - \chi_{js} \phi_j, \quad j \neq i, \quad (5.1)$$

and the coupled conserved dynamics is

$$\frac{\partial \phi_i}{\partial t} = \nabla \cdot \left(\sum_j M_{ij} \nabla \mu_j \right), \quad \mu_i = \mu_i^{\text{bulk}} - \kappa_i \nabla^2 \phi_i, \quad (5.2)$$

with M a symmetric positive-definite Onsager mobility matrix (its off-diagonal couples the two solute fluxes). This is Chapter 2 promoted to two composition fields.

Coefficients. Three interaction parameters: $\chi_{12} = 3.5$ (solute–solute; large $\Rightarrow$ the two solutes demix), $\chi_{1s} = 1$ and $\chi_{2s} = 0.6$ (solute–solvent). The mobility matrix sets the time scale and the relative solute speeds; κ_i set interface widths. Real blends map these to the χ ’s in `materials/materials.yaml` (e.g. the P3HT:PCBM and PDPP5T:PCBM systems).

5.2 The Gibbs triangle

Each node has a composition (ϕ_1, ϕ_2, ϕ_s) summing to one — a point in the 2-simplex, drawn as a point in an equilateral triangle (barycentric coordinates; solvent, ϕ_1, ϕ_2 at the three vertices). The bulk *average* is conserved and never moves; the *cloud* of local compositions starts as a tight blob at the initial blend and, as the system demixes, opens up along a tie-line toward the two coexisting phases. Watching that cloud open *is* watching ternary phase separation.

5.3 Run it

Run it.

```
cd physics/04_ternary_ch_phase_diagram
python run.py # ternary spinodal quench
```

It marches a ternary quench from a homogeneous blend $(\phi_1, \phi_2) = (0.35, 0.35)$ and reports how the composition cloud spreads and where the two coexisting phases land. Compare with `EXPECTED.md` and the Gibbs-triangle figure.

What you should see. The composition spread grows from 0.0201 (a tight initial blob) to 0.2080 as the blend separates into two phases at $(0.165, 0.519)$ and $(0.535, 0.181)$ (in (ϕ_1, ϕ_2)). The mean composition stays pinned to solver tolerance ($|\Delta\phi| \leq 5.5 \times 10^{-4}$): conservation fixes the *centroid* of the cloud while it opens along the tie-line.

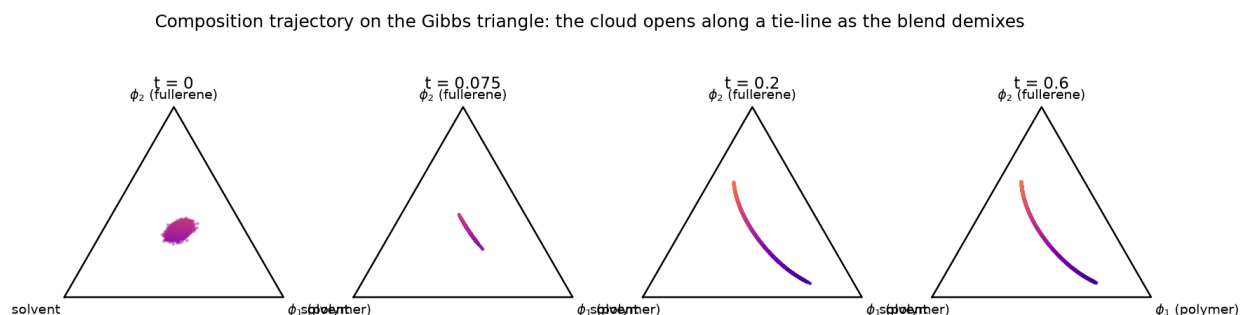


Figure 5.1: Composition trajectory on the Gibbs triangle: the local- composition cloud (one point per mesh node) opens from the initial blend along a tie-line toward the two coexisting phases. The mean (centroid) is conserved.

Final ternary morphology (the two solute fields)

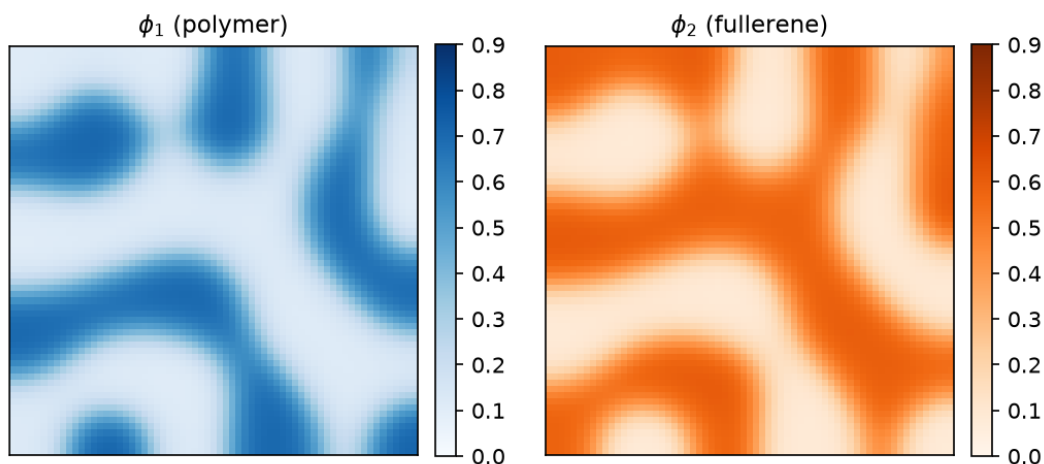


Figure 5.2: The two solute fields of the final ternary morphology.

Measured self-check.

composition spread (std ϕ_1): start $\rightarrow$ end	0.0201 $\rightarrow$ 0.2080
coexisting phase A (ϕ_1, ϕ_2)	(0.165, 0.519)
coexisting phase B (ϕ_1, ϕ_2)	(0.535, 0.181)
mean drift $ \Delta\phi $	5.5×10^{-4}

5.4 Code walk-through

1. The ternary stepper. `TernaryCHStepper(dm, chi=(...), M=(...), kappa=(...))` builds the four-field $(\phi_1, \mu_1, \phi_2, \mu_2)$ system; `set_initial(f1, f2)` seeds the two compositions.

2. Barycentric projection. `bary_to_xy(phi1, phi2)` maps a composition to triangle coordinates; scattering every node gives the cloud of Fig. 5.1.

3. Tie-line readout. Splitting the final field at the median ϕ_1 and averaging each half estimates the two coexisting phase compositions (the tie-line endpoints).

5.5 Explore on your own

Q5.5.1 Increase χ_{12} (make the two solutes more immiscible). How do the tie-line endpoints move on the triangle, and does the cloud open further? Where is the spinodal boundary for this blend?

Q5.5.2 Change the solvent selectivity: make $\chi_{1s} \neq \chi_{2s}$. Which solute segregates preferentially, and how does the tie-line *tilt* on the triangle?

Q5.5.3 Shift the initial blend off the symmetric point (e.g. solute- rich). Does the morphology change from bicontinuous to droplet, and does the cloud still open along the same tie-line?

Q5.5.4 The off-diagonal mobility M_{12} couples the two fluxes. Set it to zero and compare the coarsening: does the morphology or only the rate change?

Q5.5.5 Map a real system from `materials.yaml1` (say P3HT:PCBM) onto $(\chi_{12}, \chi_{1s}, \chi_{2s})$ and predict, from the triangle, which two phases should coexist. Then run it and check.

Chapter 6

Evaporation: the drying film

Key idea. A film is cast from solution and *dries*: solvent leaves the top surface, the film thins, and the solutes concentrate until they demix. Evaporation is the *clock* of morphology formation — how fast the solvent leaves sets how much time the blend has to phase- separate before it vitrifies.

6.1 The model

We add a moving, evaporating top surface to the ternary system of Chapter 5 (the Landau-mapped moving frame of the Wodo–Ganapathysubramanian film formulation). The computational box $[0, 1]^2$ represents a physical strip of shrinking height $h(t)$. The solvent flux at the top drives the frame:

$$K = \sum_i k_{e,i} \overline{\phi_i^{\text{top}}} \geq 0, \quad \frac{\partial h}{\partial t} = -K, \quad (6.1)$$

and each conserved solute gains a frame-advection term plus a surface- enrichment flux, arranged so the solute *content* $h \int \phi_i$ is conserved exactly per step (only the solvent leaves). The bulk dynamics is the ternary Cahn–Hilliard of Chapter 5.

The evaporation rate (Biot number). k_e is the nondimensional evaporation rate of the solvent — the Biot number $\text{Bi} = k_e h_0 / D_s$ in the wodo convention. Larger k_e = more volatile solvent = faster drying. It is *the* processing knob of solution-cast organic electronics. Different solvents map to different k_e ; spin speed and temperature also move it (see the PDPP5T:PCBM Biot values in `materials/materials.yaml`).

6.2 Run it

Run it.

```
cd physics/05_evaporation
python run.py           # sweep the evaporation rate
```

It dries the same dilute ternary film at several k_e and prints the drying time, final height, and morphology domain scale. Compare with `EXPECTED.md` and the figures.

What you should see. Every film dries (the mean solvent fraction ϕ_s falls, the height h shrinks), but faster: at $k_e = 0.6$ the film reaches the drying cutoff by $t \approx 2.87$ (final $\phi_s = 0.099$, $h = 0.488$), while at $k_e = 0.15$ it lingers to $t \approx 6.00$. And the morphology depends on the rate: faster drying freezes a *finer* pattern (domain scale 0.40 cells at $k_e = 0.6$ vs 0.60 at $k_e = 0.15$) because the blend has less time to coarsen before it vitrifies.

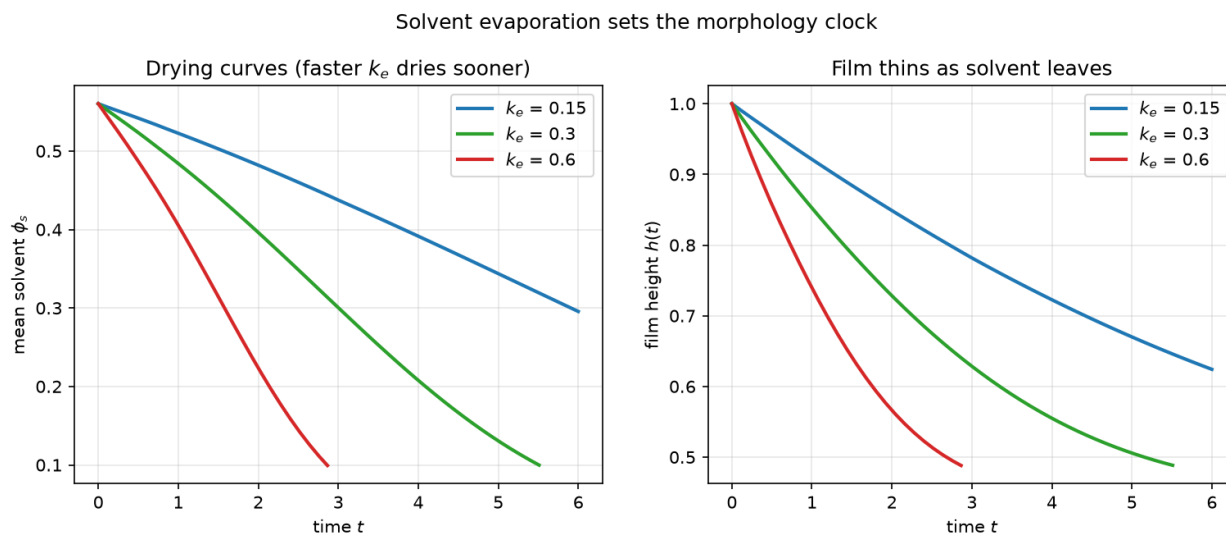


Figure 6.1: Drying curves (left, mean solvent fraction) and film height (right) for several evaporation rates. Larger k_e dries sooner and thins faster.

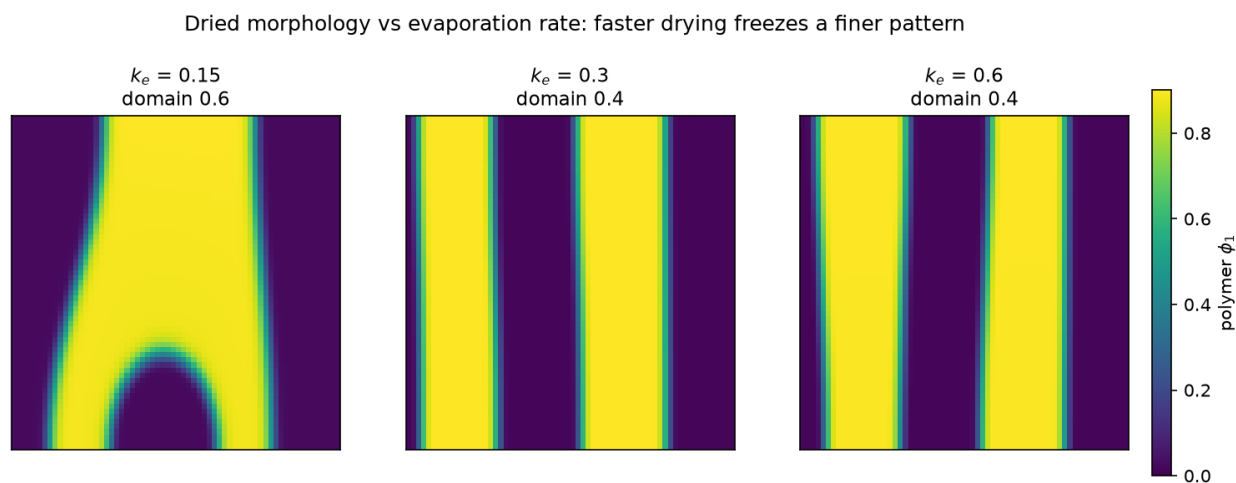


Figure 6.2: Dried morphology (the polymer field) vs evaporation rate. Faster drying freezes a finer, more kinetically trapped morphology; slower drying gives phase separation time to coarsen.

Measured self-check.

	slow ($k_e = 0.15$)	fast ($k_e = 0.6$)
drying time t	6.00	2.87
final solvent ϕ_s	0.296	0.099
final height h	0.624	0.488
domain scale (cells)	0.60	0.40

6.3 Code walk-through

1. Film mode. A single constructor argument turns on the moving frame:

```
st = MultiPhaseStepper(dm, M=2, K=0, ..., film=dict(k_e=0.1, h0=1.0))
```

k_e is the solvent evaporation rate, $h0$ the initial height. The stepper carries h_{curr} , updated $-\Delta t K$ on each accepted step.

2. The drying march. `march(..., phis_stop=0.05, h_min=0.15)` adds film stop criteria (dryness / height floor) and the evaporation Δt -cap that bounds the height decrement per step.

3. The domain scale. `_domain_scale` reads the characteristic lateral wavelength from the first spectral moment of the polymer field — a single number for “how fine is the morphology”.

6.4 Explore on your own

Q6.4.1 Push k_e higher and lower by another factor. Does the domain scale keep shrinking with rate, or does it saturate? What sets the finest morphology you can freeze in?

Q6.4.2 Change the solvent selectivity (χ_{1s} vs χ_{2s}). Does a solvent that likes one solute more change which component enriches at the drying surface?

Q6.4.3 The drying curve $\phi_s(t)$ is roughly linear early (constant-rate period) then bends. Explain the bend in terms of the surface composition feeding back into $K = \sum_i k_{e,i} \phi_i^{\text{top}}$.

Q6.4.4 Combine this with Chapter 4: add a substrate wall energy and dry the film. Does evaporation-driven enrichment at the top compete with substrate enrichment at the bottom? Which wins?

Q6.4.5 Two solvents of different volatility can be modelled as different k_e . Predict how a fast vs a slow solvent changes the vertical stratification, then test it.

Chapter 7

Allen–Cahn crystallization

Key idea. Crystallinity is a *non-conserved* order parameter — crystal can be created, unlike composition — so it obeys *Allen–Cahn* (non-conserved gradient flow), not Cahn–Hilliard. A crystal grows below the melting point and melts above it; the way its fraction advances in time is the Avrami (JMAK) law.

7.1 The model

Let $\psi \in [0, 1]$ be the local crystallinity of a crystallizable species. Its non-conserved gradient flow is

$$\frac{\partial \psi}{\partial t} = -L_\psi \left[\frac{\partial f}{\partial \psi} - \varepsilon^2 \nabla^2 \psi \right], \quad (7.1)$$

with the crystallization free energy (the 2310.11844 “r14” form, PCBM-class energetics)

$$f_{\text{cr}} = \phi [q(\psi) \Delta\sigma + p(\psi) \text{drive}], \quad q = \psi^2(1 - \psi)^2, \quad p = \psi^2(3 - 2\psi), \quad (7.2)$$

and the Turnbull undercooling driving force

$$\text{drive} = \Delta h (T/T_m - 1). \quad (7.3)$$

$q(\psi)$ is a double-well barrier separating amorphous ($\psi = 0$) from crystalline ($\psi = 1$); $p(\psi)$ interpolates the driving; L_ψ sets the growth rate; ε^2 the crystal-interface width.

The melting point sets the sign. $\text{drive} \propto (T/T_m - 1)$ is *negative* for $T < T_m$ (so $\psi \rightarrow 1$ lowers f : the crystal *grows*) and *positive* for $T > T_m$ (the crystal *melts*). $T_m = 558$ K here (materials.yaml, PCBM class). Δh is the crystallization enthalpy, $\Delta\sigma$ the barrier height, L_ψ the kinetic prefactor, ε^2 the gradient penalty (crystal interface width $\sim \sqrt{\varepsilon^2}$).

7.2 Growth kinetics: the Avrami law

With athermal nuclei placed at $t = 0$ growing at a roughly constant interface velocity v , the crystalline area of each grain grows like $(vt)^2$ in 2-D before grains impinge, so the crystalline *fraction* follows the Avrami/Johnson–Mehl–Avrami–Kolmogorov (JMAK) law

$$X(t) = 1 - \exp[-(kt)^n], \quad (7.4)$$

with Avrami exponent n reporting the growth dimensionality and nucleation mode. For *point* nuclei appearing over time with 2-D growth, $n \approx 3$; for *pre-existing* 2-D nuclei, $n \approx 2$; and *finite, pre-placed* supercritical seeds (what we use, because point seeds are sub-critical and redissolve) give an effective exponent *below* 2 — the seeds start with area, so the early t^2 regime is foreshortened. We fit n from the measured $X(t)$ via the double-log linearization $\ln[-\ln(1 - X)] = n \ln t + \text{const}$ over the pre-impingement window.

7.3 Run it

Run it.

```
cd physics/06_allen_cahn_crystallization
python run.py                # grow/melt + Avrami fit
```

It seeds a single crystal below and above T_m (growth vs melting) and a multi-nucleus run whose fraction is fit to the Avrami law. Compare with `EXPECTED.md` and the figures.

What you should see. Below the melting point ($T = 333$ K) the seeded disc *grows* (crystalline area $0.0693 \rightarrow 0.1101$, a factor 1.59); above it ($T = 700$ K) it *melts* ($0.0693 \rightarrow 0.0000$). The multi-nucleus run reaches crystalline fraction $X = 0.858$ from 4 nuclei (3 grains resolved), and the fitted Avrami exponent is $n = 1.47$ — an intermediate value below the ideal $n = 2$ because the nuclei are finite pre-placed seeds (the exercises explore how the nucleation mode moves n).

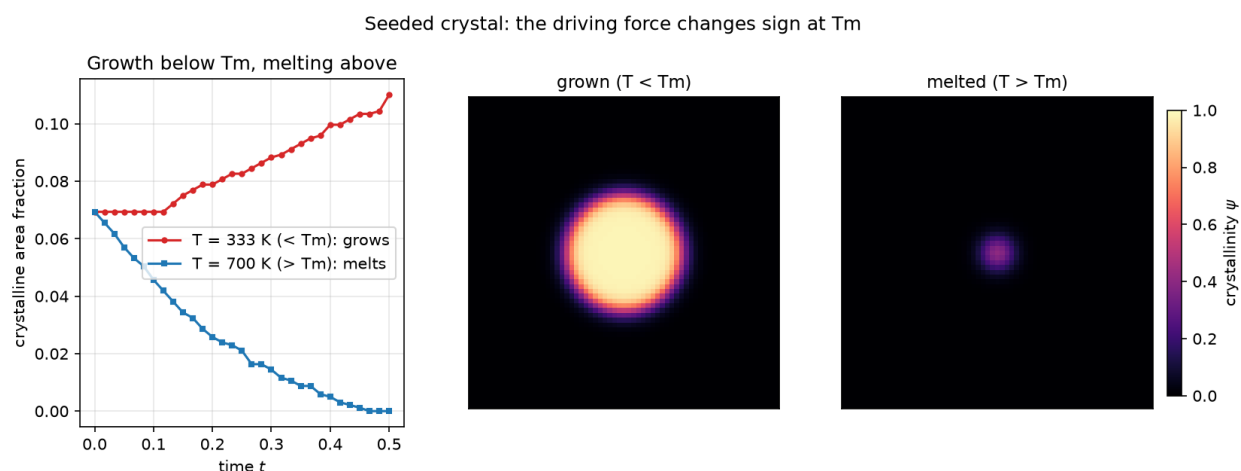


Figure 7.1: A seeded crystal grows below T_m and melts above it (left: area vs time; right: the final ψ fields). The driving force changes sign at the melting point.

Measured self-check.

grow ($T < T_m$): area start $\rightarrow$ end	0.0693 $\rightarrow$ 0.1101 (1.59 $\times$)
melt ($T > T_m$): area start $\rightarrow$ end	0.0693 $\rightarrow$ 0.0000
Avrami exponent n	1.47
crystalline fraction X_{end}	0.858
nuclei $\rightarrow$ grains resolved	4 $\rightarrow$ 3

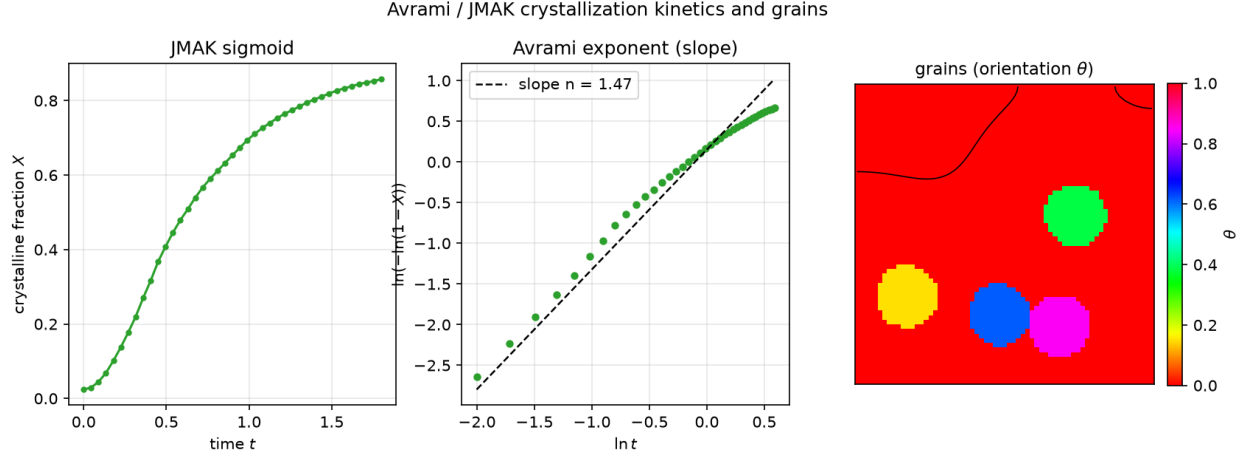


Figure 7.2: Avrami/JMAK kinetics: the sigmoidal crystalline fraction $X(t)$ (left), the double-log linearization whose slope is the Avrami exponent (middle), and the grains coloured by orientation θ with the crystal outline (right).

7.4 Code walk-through

1. The crystallizing stepper. `MultiPhaseStepper(dm, M=1, K=1, ..., bulk="r14")` adds one crystallinity field ψ (and its orientation θ) to one composition. `dsig`, `dh`, `Tm`, `eps2`, `L_psi` are the coefficients above.

2. Seeding a crystal. `set_initial([phi], [psi_disc], [theta])` places a ψ disc (a tanh profile) — the athermal nucleus — with an orientation marker.

3. The Avrami fit. `_fit_avrami` linearizes $X(t)$ over the growth window ($0.05 < X < 0.9$) and reads n as the slope; the grains come from `grain_labels` (a θ -watershed).

7.5 Explore on your own

Q7.5.1 Vary the undercooling T/T_m . Does the growth velocity (and so the Avrami k) increase monotonically as you cool further below T_m , or is there a maximum? Why might very deep undercooling slow growth?

Q7.5.2 Change the number of pre-placed nuclei. Does the Avrami exponent n stay near 2 (fixed nuclei) or move toward 3 (if nucleation kept happening during growth)? What does n tell you about the nucleation mode?

Q7.5.3 Vary ε^2 and measure the crystal interface width. Confirm the $\sqrt{\varepsilon^2}$ scaling, and relate the width to the mesh spacing — when is the crystal interface under-resolved?

Q7.5.4 Give two seeds *different* orientation markers θ and let them grow into contact. Do they merge or form a boundary? (This is the coupled orientation dynamics of Chapter 8.)

Q7.5.5 The crystal expels the amorphous component as it grows (the ϕ field). Measure the composition inside vs outside the crystal — this is the crystallization-driven demixing that Chapter 8 makes central.

Chapter 8

Coupled Cahn–Hilliard + Allen–Cahn

Key idea. The real morphology problem is *coupled*: composition (Cahn–Hilliard) and crystallinity (Allen–Cahn) evolve together and feed back. The signature of the coupling is *crystallization-driven demixing* — as a species crystallizes it expels the others, sharpening a composition pattern that phase separation alone would not produce.

8.1 The model: the four-fold χ

With crystallinity, the interaction between two species depends on whether each is amorphous or crystalline, so the single Flory χ becomes *four* matrices:

$$\chi_{\text{eff}}(i, j) = \chi_{aa} + \psi_i^2 \chi_{ca} + \psi_j^2 \chi_{ac} + \psi_i^2 \psi_j^2 \chi_{cc}, \quad \chi_{ca} = \chi_{ac}^\top, \quad (8.1)$$

coupling the Cahn–Hilliard composition fields to the Allen–Cahn crystallinity fields. Making χ_{ca} *larger* than χ_{aa} means “a crystal of species i dislikes amorphous species j more than two amorphous phases dislike each other” — so crystallizing i pushes j out. That is the thermodynamic origin of crystallization-driven demixing.

Coefficients. The bulk Cahn–Hilliard χ_{aa} and the crystal-contact χ_{ca} (with $\chi_{ca} > \chi_{aa}$ for expulsion), plus the crystallization energetics of Chapter 7 (Δh , $\Delta\sigma$, T , T_m , ε^2 , L_ψ). The *demixing metric* is the purity contrast: the mean composition of the crystallizing species *inside* the crystal ($\psi > 0.5$) minus *outside*.

8.2 Run it

Run it.

```
cd physics/07_coupled_ch_ac
python run.py                # crystallization ON vs OFF, then the (M,K)
    ladder
```

It runs $(M, K) = (2, 1)$ with crystallization ON and OFF, prints the demixing purity contrast, and repeats the ON case at $(3, 1)$ and $(3, 2)$. Compare with `EXPECTED.md` and the figures.

What you should see. With crystallization ON, the crystallizing species concentrates strongly inside the crystal (inside 0.852 vs outside 0.436; contrast +0.416). With crystallization OFF (the

same blend, $K = 0$) the composition only demixes by ordinary Cahn–Hilliard, a much smaller contrast $+0.000$ — crystallization amplifies the demixing by $21\times$. The same machinery scales up the ladder: $(3, 1)$ gives contrast $+0.391$ and $(3, 2)$ gives $+0.170$.

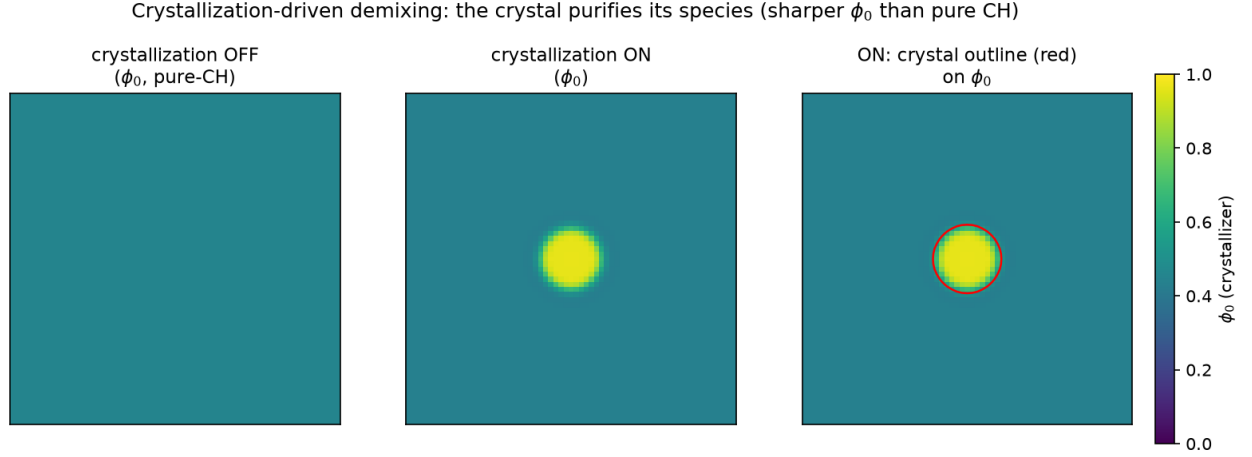


Figure 8.1: Crystallization-driven demixing at $(M, K) = (2, 1)$. Left: crystallization off (pure Cahn–Hilliard composition). Middle/right: crystallization on — the crystal (red outline) purifies its species, a sharper composition pattern than phase separation alone produces.

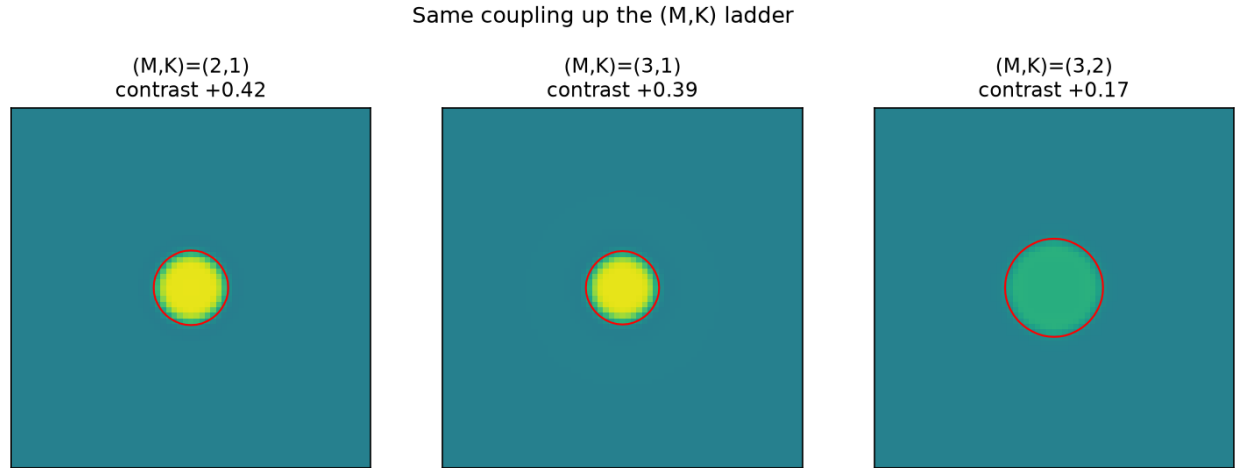


Figure 8.2: The same coupled model up the (M, K) ladder: $(2, 1) \rightarrow (3, 1) \rightarrow (3, 2)$. Adding species and crystallizable phases is a change of two integers; the physics (crystallization-driven demixing) carries over.

Measured self-check.

$(2, 1)$ crystallizer ϕ_0 : inside $\rightarrow$ outside crystal	$0.852 \rightarrow 0.436$
demixing contrast: ON vs OFF	$+0.416$ vs $+0.000$
amplification (ON/OFF)	$21\times$
contrast at $(3, 1)$ / $(3, 2)$	$+0.391$ / $+0.170$

8.3 Code walk-through

1. Four χ matrices. `_chis` builds the amorphous χ_{aa} (all pairs) and the stronger crystal-contact χ_{ca} (a crystallizing species vs the others); passing them to `MultiPhaseStepper(chi_aa=..., chi_ac=..., chi_ca=...)` turns on the coupling.

2. ON vs OFF control. The same call with $K = 0$ drops the crystal terms, isolating the pure-Cahn–Hilliard demixing — the control that makes the crystallization contribution measurable.

3. The purity contrast. The demixing metric splits the field by the crystal mask $\psi > 0.5$ and differences the mean composition, inside vs outside.

8.4 Explore on your own

Q8.4.1 Sweep χ_{ca} from χ_{aa} (no extra crystal repulsion) upward. At what point does crystallization-driven demixing switch on, and how does the purity contrast scale with $\chi_{ca} - \chi_{aa}$?

Q8.4.2 Turn crystallization on *after* the blend has partly demixed (delay the seed). Does the crystal template on the existing composition pattern, or reshape it? What does this say about processing order (phase-separate then crystallize vs simultaneously)?

Q8.4.3 At (3,2) two species crystallize. Do their crystals compete for the same regions, or segregate? How does the second crystallizable phase change the amorphous matrix left behind?

Q8.4.4 Compare the coarsening rate with and without crystallization. Does the growing crystal *pin* the composition pattern (slower coarsening), and why would a device designer want that?

Q8.4.5 The contrast is a purity measure. Relate it to what a solar cell wants (pure domains for transport, but a bicontinuous network for charge separation). Is more contrast always better?

Chapter 9

Thermal noise and nucleation

Key idea. Where do the crystal seeds of Chapter 7 come from? From thermal *fluctuations*. An undercooled melt is *metastable*: $\psi = 0$ sits behind a free-energy barrier, and the crystal cannot form until a fluctuation pushes a small region over that barrier — classical nucleation. Here the noise is *physics*, and its amplitude ($k_B T$) sets the nucleation rate.

9.1 The model

We add the physical (fluctuation–dissipation) noise to the Allen–Cahn crystallinity equation:

$$\frac{\partial \psi}{\partial t} = -L_\psi \left[\frac{\partial f}{\partial \psi} - \varepsilon^2 \nabla^2 \psi \right] + \xi, \quad \langle \xi(\mathbf{x}, t) \xi(\mathbf{x}', t') \rangle = 2L_\psi k_B T \delta(\mathbf{x} - \mathbf{x}') \delta(t - t'), \quad (9.1)$$

an FDT-normalized Gaussian forcing with `noise_psi`² = $k_B T$ in the nondimensional units. The *p1* bulk free energy has a genuine nucleation barrier at $\psi = 0$ ($\partial f / \partial \psi = \phi^2 N^{-1} \Delta \sigma > 0$ there), so with *no* noise ψ stays 0 forever; with noise, rare fluctuations climb the barrier, form supercritical nuclei, and those grow.

The physical seed. In Chapter 2 the random seed was a *numerical* convenience (a fixed RNG seed; the statistics were seed-independent). *Here* the noise is a first-class physical object: its amplitude is $k_B T$ and it sets the nucleation *rate*. Nucleation is a rare-event, barrier-crossing process, so the rate scales like $\exp(-\Delta F^* / k_B T)$ — a small increase in noise amplitude produces a large increase in how much crystal forms by a fixed time. (BDF1 only: the brick asserts noise off under BDF2, the stochastic weak order being a recorded scope limit.)

9.2 Run it

Run it.

```
cd physics/08_noise_nucleation
python run.py                # sweep the FDT noise amplitude
```

It runs the same undercooled melt (ψ starts at 0, *no* seed) at several noise amplitudes and prints the final crystalline fraction, the grain count, and the induction time. Compare with `EXPECTED.md` and the figures.

What you should see. With zero noise the melt stays amorphous ($X = 0.000$: no nucleation). Increasing the FDT amplitude nucleates crystal — by the largest amplitude (`noise_psi` = 0.12) the melt reaches $X = 0.516$ in 25 distinct grains. The onset is threshold-like: nucleation switches on between the amplitudes, reflecting the exponential barrier-crossing rate. The noise is the physical seed.

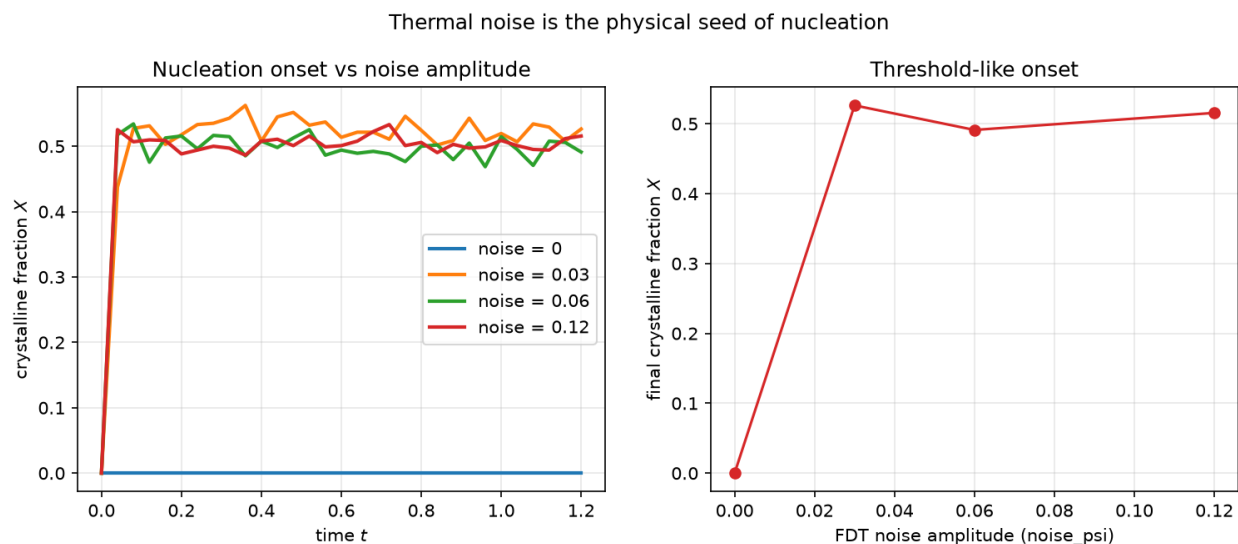


Figure 9.1: Left: crystalline fraction $X(t)$ for several FDT noise amplitudes — zero noise never nucleates; larger noise nucleates sooner. Right: the final crystalline fraction vs noise amplitude, a threshold-like onset.

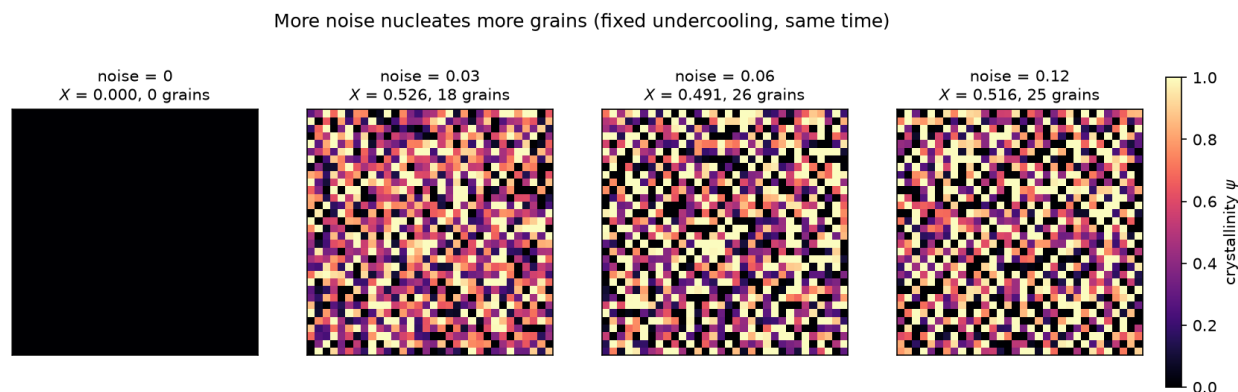


Figure 9.2: Final crystallinity fields across the noise sweep: more noise nucleates more grains (same undercooling, same time).

Measured self-check.

	zero noise	max noise (0.12)
final crystalline fraction X	0.000	0.516
grains nucleated	0	25

Nucleation onset (first amplitude with $X > 0.02$): 0.03.

9.3 Code walk-through

1. Turn on the FDT noise. Two constructor arguments:

```
st = MultiPhaseStepper(dm, M=1, K=1, ..., timestep="bdf1",
                        noise_psi=0.06, noise_seed=17)
```

`noise_psi` is $\sqrt{k_B T}$ (the amplitude), `noise_seed` fixes the fluctuation realization for reproducibility.

2. No seed. `set_initial([phi], [zeros], [zeros])` starts from a uniform amorphous melt — the crystal must nucleate on its own.

3. The sweep. `sweep` runs several amplitudes on the same melt and reads X_{end} , the grain count (`grain_labels`), and the induction time.

9.4 Explore on your own

Q9.4.1 The nucleation rate should scale like $\exp(-\Delta F^*/k_B T)$. Plot $\ln(\text{grains})$ (or $\ln X$) against $1/\text{noise_psi}^2$. Is it roughly linear, and what does the slope estimate about the barrier height?

Q9.4.2 Deepen the undercooling (lower T) at fixed noise. Does the induction time fall? Relate this to how the barrier ΔF^* depends on the driving force.

Q9.4.3 Change only the `noise_seed` at fixed amplitude. Which is reproducible — the exact grain *positions*, or the statistics (grain count, X)? Connect this to the seed discussion in Chapter 2: here the seed is physical, so a different seed is a different (equally valid) sample of the same fluctuating system.

Q9.4.4 Is there an amplitude so small that nothing nucleates within the run, yet the melt is still undercooled? What does that say about metastability and the difference between *unstable* (spinodal) and *metastable* (nucleation) states?

Q9.4.5 Combine with Chapter 7: once a fluctuation nucleates a supercritical crystal, does it grow at the same rate as a hand-placed seed of the same size? Should it?

Chapter 10

Evaporation-induced crystallization

Key idea. This is the whole story, at tutorial scale: a *wet* film *dries* (Ch. 6), the concentrating blend *phase-separates* (Ch. 2–5), and once the crystallizable species is concentrated enough its crystals *nucleate and grow* (Ch. 7) coupled back to composition (Ch. 8). The arc *wet film* $\rightarrow$ *phase separation* $\rightarrow$ *nucleation* $\rightarrow$ *crystalline film* is how a real solution-cast organic solar cell forms.

10.1 The model

We run $(M, K) = (2, 1)$ in film mode (Ch. 6): a crystallizable small molecule (species 0), a polymer (species 1), and an evaporating solvent. The r14 crystallization bulk carries a crystal-contact penalty χ_{ca} , which gives a *solubility*: crystallization is forbidden below a local small-molecule fraction ϕ^* set by the undercooling versus χ_{ca} . So the *same* crystal seed behaves oppositely depending on *when* it is introduced:

- implanted in the **wet** film (solvent-rich, ϕ_0 below ϕ^*) it *dissolves*;
- implanted **mid-drying** (drying has concentrated ϕ_0 above ϕ^* , and continuing solvent loss keeps deepening the quench) it *grows* to a crystalline film.

That ordering — crystallization strictly *after* significant solvent loss — is the physics.

Seeding is physical and delicate (the S3b campaign lessons). The implanted embryo radius r_0 must clear the Gibbs–Thomson critical radius r^* with margin (sub-critical embryos redissolve), and the embryo amplitude must be near 1 (a half-amplitude embryo halves the bulk driving and doubles r^*). We use r_0 with margin and $\psi \approx 0.95$, and read the *terminal* crystalline state, not a mid-growth snapshot.

Tutorial simplification (recorded honestly). The production configuration (`tests/test_multiphase_s3.py`) uses the Vignes composition-singular mobility, whose 3-decade liquid $\rightarrow$ solid drop is faithful but stiff. Here we use a *constant* Onsager mobility (with the same cuDSS direct solver); the qualitative arc is unchanged, the quantitative drying-front sharpness is softened.

10.2 Run it

Run it.

```
cd physics/09_evaporation_crystallization
python run.py # wet-implant vs dry-implant
```

It dries the film and implants seeds either early (wet) or mid-drying, and prints the drying state at implant and the terminal crystalline area for each. Compare with EXPECTED.md and the figures.

What you should see. Seeds implanted in the wet film ($\phi_s = 0.836$) *dissolve* (crystalline area $0.3116 \rightarrow 0.0000$, $\psi_{\max} = 0.00$). The same seeds implanted mid-drying ($\phi_s = 0.633$) *grow* to a crystalline film (area $0.3116 \rightarrow 1.0000$, $\psi_{\max} = 0.99$). The difference is entirely *when* they were introduced relative to the drying — evaporation-induced crystallization.

The evaporation-induced crystallization arc: wet film -> phase separation -> nucleation -> crystalline film

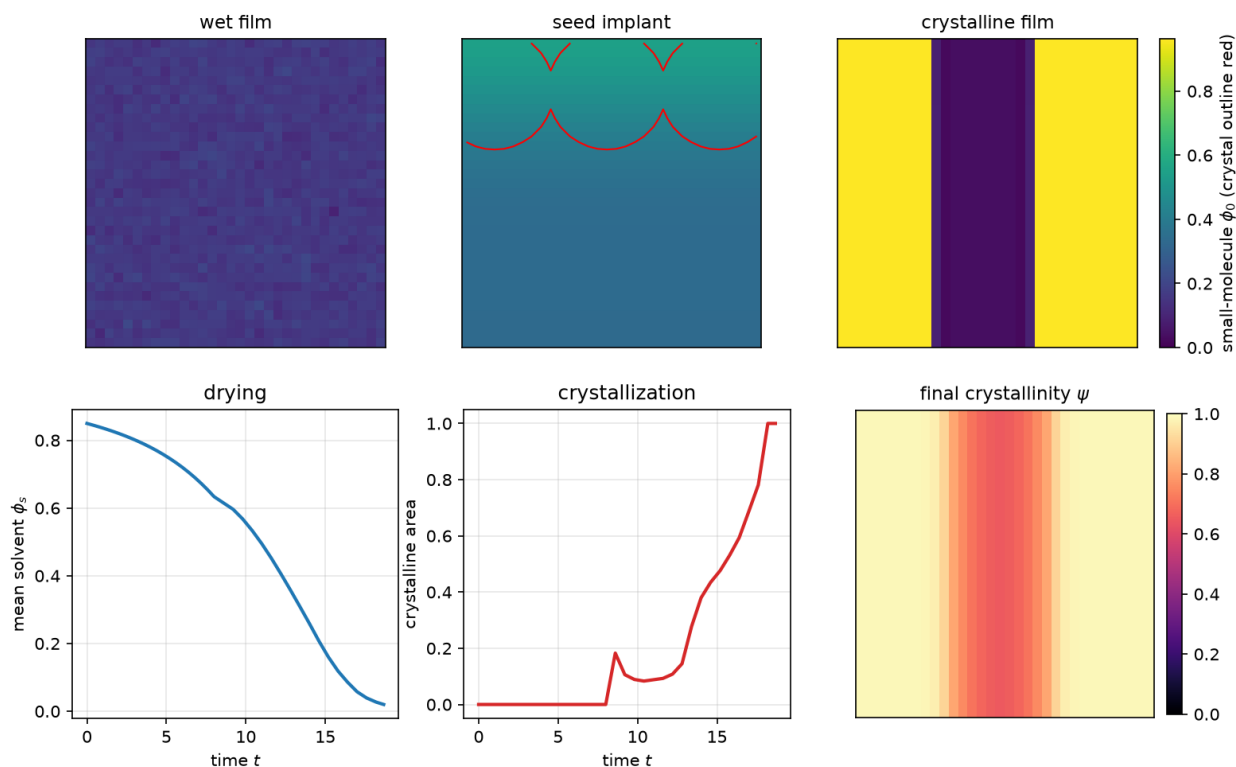


Figure 10.1: The arc (dry-implant run): top — wet film, seed implant, and the final crystalline film (small-molecule field, crystal outline in red); bottom — the drying curve $\phi_s(t)$, the crystalline area growth, and the final crystallinity ψ .

Measured self-check.

Same seeds: dissolve when wet, grow when dry (evaporation-induced)

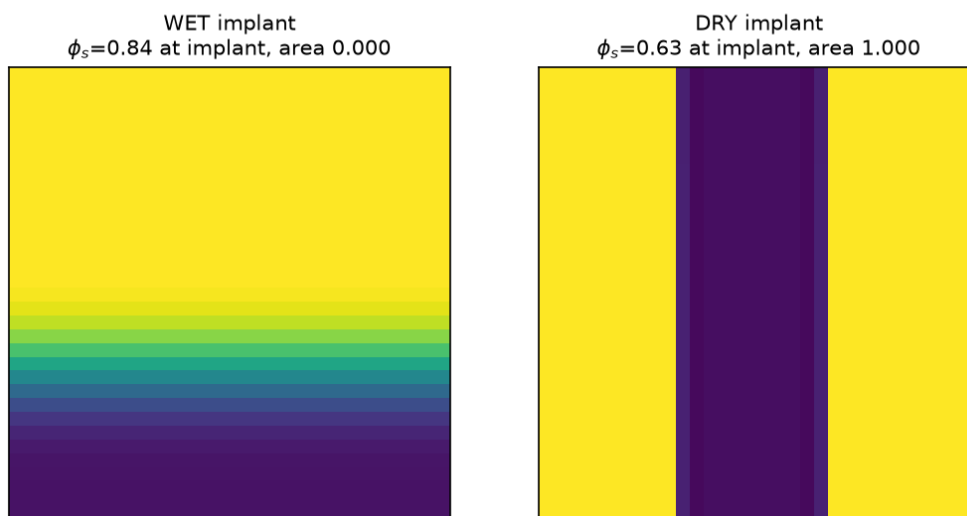


Figure 10.2: The same seeds, implanted wet (left, dissolve) vs mid-drying (right, grow). The solubility set by the drying decides the fate.

	wet implant	dry implant
ϕ_s at implant	0.836	0.633
crystalline area: start $\rightarrow$ end	0.3116 $\rightarrow$ 0.0000	0.3116 $\rightarrow$ 1.0000
$\psi_{\max}$ (terminal)	0.00	0.99

10.3 Code walk-through

1. The drying, crystallizing film. `make_stepper` builds the (2,1) film-mode stepper (r14, cuDSS) and asserts the crystal energetics actually reached it — the missing-splat guard from the S3b campaign (a dropped keyword silently defaulted $T_m = 1$ and turned the driving into a huge *melting* force).

2. Implant and march. `run_arc` marches the drying film to the implant time, calls `implant` (a ψ disc with margin over r^* at the small-molecule-richest sites), and marches on to the terminal, dried state.

3. Read the terminal state. The fate is read from the terminal crystalline area and $\psi_{\max}$, not a transient — the robust observables the S3b gates lock.

10.4 Explore on your own

Q10.4.1 Sweep the implant time from wet to dry. Is there a sharp crossover (a critical ϕ_s) between dissolution and growth, and does it match the r14 solubility $\phi^* = 1 - \text{drive}/\chi_{ca}$?

Q10.4.2 Change the evaporation rate k_e (Ch. 6). Does faster drying reach the crystallization window sooner, and does it change the final crystalline morphology (grain size, coverage)?

Q10.4.3 Vary the implant radius r_0 around the critical radius. Find the margin below which even a mid-drying seed redissolves — the Gibbs–Thomson r^* in the developing film.

Q10.4.4 Replace the hand-placed seed with FDT noise (Ch. 9): let the drying film nucleate on its own. Does crystallization still wait for the solvent to leave? What sets the number of crystals?

Q10.4.5 This is the design problem: given a target (fine, pure, bicontinuous crystalline domains), which knobs — k_e , χ_{ca} , undercooling, blend ratio — would you turn, and which fight each other? (The Differentiable track answers this with gradients.)

Part II

Computation

Chapter 11

Convergence: basis order and time integration

Key idea. A discretization is only trustworthy if it *converges* at the rate the theory predicts. The method of manufactured solutions (MMS) measures that rate directly: plug a known solution into the equations, add the residual as a source, and check that the error falls like h^{p+1} in space and Δt^q in time.

This concept walks through the Cahn–Hilliard convergence gates that ship with the solver: $p = 1$ vs $p = 2$ spatial order, and BDF1 vs BDF2 temporal order, each measured against a manufactured solution. It connects to Physics Chapter 2 (the same brick) and to the adaptive-stepping concept (Chapter 13: why variable-coefficient BDF2 matters once Δt changes).

11.1 The two kinds of error

Every result from this simulator is wrong. The question is only *how* wrong, and whether the error shrinks — at the rate the theory promises — as you refine. A finite-element discretization has two independent discretization errors:

- a **spatial** error from representing $c(\mathbf{x})$ in a finite basis on a mesh of size h ; for degree- p elements the L^2 error scales as h^{p+1} ;
- a **temporal** error from advancing in finite steps Δt ; for a scheme of order q it scales as Δt^q .

The exponents $p+1$ and q are *predictions*. Measuring them is the single most important correctness check in scientific computing: a code can run, conserve mass, and look physical while quietly converging one order too slow because a term is missing from the weak form. This concept measures both.

11.2 Spatial order: the method of manufactured solutions

The trouble with checking a solver against a “true” answer is that PDEs rarely have closed-form solutions. The *method of manufactured solutions* (MMS) turns the problem around: *pick* a smooth field, then compute what source term would make it an exact solution, and feed that source to the solver. Now you know the exact answer everywhere, so the error is measurable.

We manufacture the composition and the chemical potential *independently* (the cleanest choice

— it avoids taking a Laplacian of the cubic $f'(c)$ by hand):

$$c^* = \cos(\pi x) \cos(\pi y) e^{-t}, \quad \mu^* = \sin(\pi x) \sin(\pi y) e^{-t}. \quad (11.1)$$

Substituting (c^*, μ^*) into the mixed Cahn–Hilliard system (2.2) leaves a residual; we *add it back* as two source terms f_c, f_m so the pair is an exact solution:

$$f_c = c_t^* - M \nabla^2 \mu^* = -c^* + 2\pi^2 M \mu^*, \quad (11.2)$$

$$f_m = \mu^* - f'(c^*) + \kappa \nabla^2 c^* = \mu^* - ((c^*)^3 - c^*) - 2\pi^2 \kappa c^*. \quad (11.3)$$

Because the sources are built from the exact field, everything *except* the discretization cancels: the measured L^2 error is pure discretization error. We impose c^*, μ^* as strong (Dirichlet) data on the boundary (the manufactured field is not no-flux), then halve h and watch the error fall.

Reading a convergence order. If the error is $E(h) \approx C h^k$, then between two meshes with h and $h/2$, $\log_2(E(h)/E(h/2)) = k$. That ratio — the *observed order* — is what you compare against the theoretical $p + 1$. It is far more diagnostic than any single error value: the *slope* is the physics of the method; the *intercept* C is just problem-dependent.

11.3 Temporal order: self-convergence

For the time error we do not need a manufactured source. We freeze the mesh, march the same smooth initial blend with a shrinking Δt , and compare each run against a very fine- Δt reference march on the identical mesh — so only the time error remains. Backward Euler (**BDF1**) is first order; the two-step **BDF2** is second order. At constant Δt , BDF2 runs its textbook coefficients $\frac{3}{2}c^{n+1} = 2c^n - \frac{1}{2}c^{n-1} + \Delta t(\cdots)$; Chapter 13 returns to what happens when Δt is allowed to *vary* (and why the coefficients must then change).

11.4 Run it

Run it.

```
cd computational/01_convergence
python run.py                # spatial + temporal studies
```

It runs the MMS study at $p = 1$ and $p = 2$, the temporal study for BDF1 and BDF2, and prints the observed orders with a PASS/FAIL gate check. Compare with EXPECTED.md and the figures below.

Measured self-check (your `run.py` should reproduce these to a couple of significant figures; the observed *orders* are the load-bearing quantities).

study	discretization	error (coarse $\rightarrow$ fine)	observed order
spatial (MMS)	$p = 1$	$2.9 \times 10^{-3} \rightarrow 1.8 \times 10^{-4}$	2.00
	$p = 2$	$2.0 \times 10^{-4} \rightarrow 2.5 \times 10^{-5}$	3.00
temporal	BDF1	$1.1 \times 10^{-3} \rightarrow 2.9 \times 10^{-4}$	0.98
	BDF2	$4.5 \times 10^{-5} \rightarrow 2.3 \times 10^{-6}$	2.13

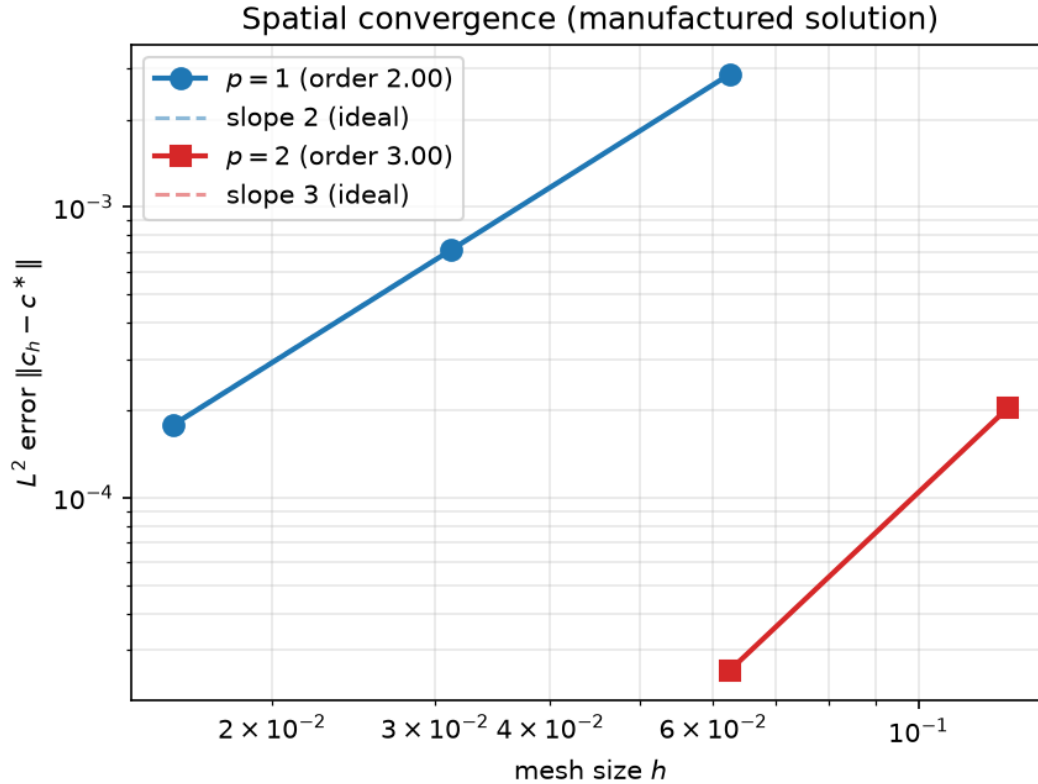


Figure 11.1: Spatial convergence under mesh refinement. The L^2 error against the manufactured solution falls along the ideal slope $p+1$: order 2.00 for $p = 1$ and 3.00 for $p = 2$ (dashed = ideal h^{p+1}).

Expected: spatial order $p + 1$ (2 and 3); temporal order 1 (BDF1) and 2 (BDF2).

11.5 Code walk-through

The study lives in `convergence.py`, which drives the production brick `src/diffsim/physics/cahn_hilliard.py` — the same brick as Physics Chapter 2.

1. The manufactured solution and its sources.

```
def c_star(x, t): return cos(pi x) cos(pi y) e^{-t}
def f_c(x, t):   return -c_star + M*2*pi^2 * mu_star    # residual, added
back
```

`c_star`, `mu_star` are the exact fields; `f_c`, `f_m` are the source terms that make them exact. These are passed to the stepper as `fc_fn`, `fm_fn`; the boundary data goes in as `gc_fn`, `gm_fn` on the Dirichlet nodes.

2. The spatial sweep.

```
for lv in levels:                # 4, 5, 6 (p=1)
    dm, mesh, cons = build_dm(lv, p) # 2^lv cells per side
    st = CahnHilliardStepper(dm, M, kap, dt, order=2,
                             fc_fn=f_c, fm_fn=f_m, dirichlet=..., ...)
```

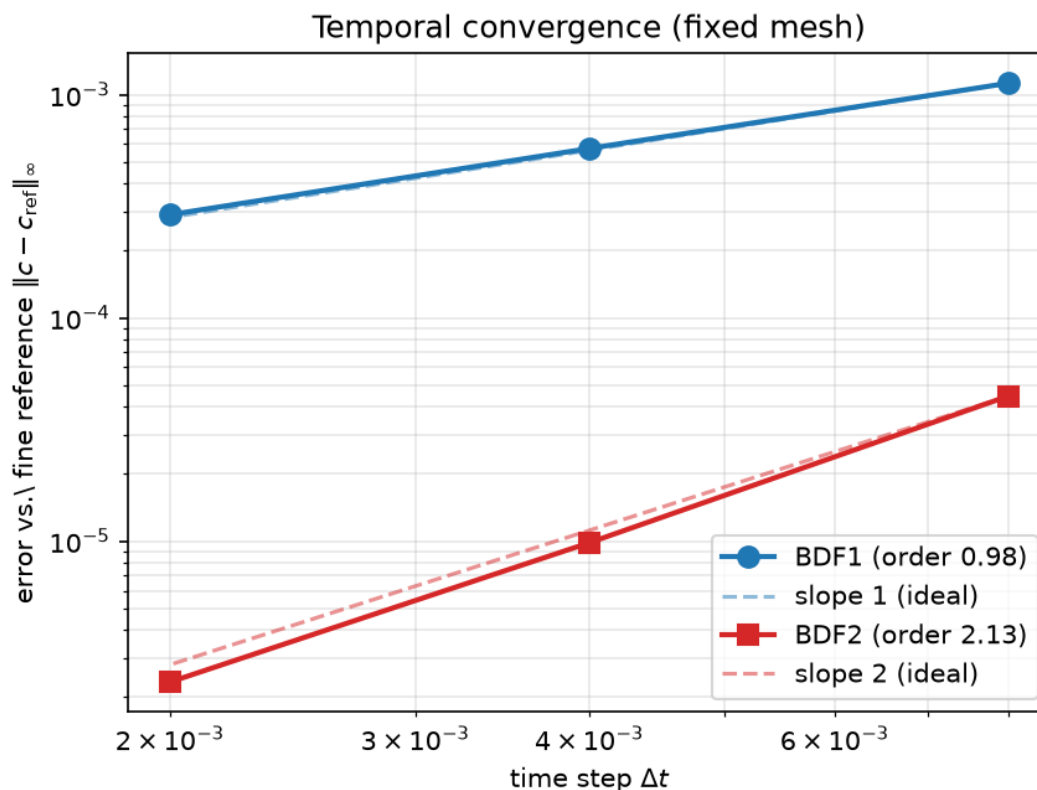


Figure 11.2: Temporal convergence at fixed mesh. Error against a fine- Δt reference march versus Δt : BDF1 tracks the slope-1 line (order 0.98), BDF2 the slope-2 line (order 2.13). At the same Δt , BDF2 is over an order of magnitude more accurate.

```
for _ in range(6): c, _ = st.step()
    errs.append(l2_error(dm, cons.T @ c, lambda x: c_star(x, st.t)))
order = log2(errs[0] / errs[1])           # the observed order
```

`l2_error` integrates $\|c_h - c^*\|$ with the same quadrature the solver uses. The ratio of successive errors, base-2, is the order.

3. Temporal self-convergence.

```
ref = march(order=2, dt=2e-4)           # fine reference on this mesh
errs = [max|march(order, dt) - ref| for dt in (8e-3, 4e-3, 2e-3)]
```

The reference is a high-accuracy BDF2 march; each coarse march's error is measured against it, isolating the time error. `order=1` selects BDF1, `order=2` selects BDF2 — one flag in the same constructor.

11.6 Explore on your own

Q11.6.1 Add a $p = 1$ level 7 (128×128) point to the spatial sweep. Does the observed order stay at 2, or does it drift? At some resolution the error stops being dominated by the spatial term — predict which term (the fixed Δt , or floating-point round-off) takes over, and confirm it by also refining Δt .

Q11.6.2 Change the manufactured field's frequency to $\cos(2\pi x)\cos(2\pi y)$. The error *intercept* should jump (the field is harder to resolve) while the *order* stays $p + 1$. Verify both, and explain why the slope is invariant but the constant is not.

Q11.6.3 Break the solver on purpose: drop the $\kappa \nabla^2 c$ term from $\mathbf{f}_m$ (so the source no longer matches the manufactured field). What order do you now measure, and why is a wrong-but-nonzero order more dangerous than an obvious crash?

Q11.6.4 Measure BDF2's temporal order at a *coarser* reference ($\Delta t_{\text{ref}} = 10^{-3}$ instead of 2×10^{-4}). At what point does the reference stop being "exact enough" and the measured order degrade? This is the self-convergence method's one pitfall — quantify it.

Q11.6.5 The p2 spatial study uses coarser levels (3, 4) than p1 (4, 5, 6). Why can it afford to? Count the degrees of freedom at $p=2$ level 4 versus $p=1$ level 6 and compare the two errors per DOF — which discretization is more accurate for the same cost here?

Chapter 12

Boundary conditions, numerically

Key idea. A boundary condition is not an afterthought bolted onto a solved interior — it is baked into the weak form, and the *way* it is imposed decides what the simulation conserves. A no-flux (natural) boundary costs nothing to impose and conserves mass exactly; a Dirichlet boundary is imposed by replacing rows and *deliberately* breaks conservation, because it lets material cross the boundary.

This concept shows, with measured numbers, how the two boundary treatments the Cahn–Hilliard brick supports are imposed and what each one costs and means. It connects to Physics Chapter 2 (whose spinodal runs are all no-flux) and to the manufactured-solution study of Chapter 11 (which needed Dirichlet data).

12.1 Boundaries live in the weak form

Multiply the conserved dynamics (2.2) by a test function w and integrate by parts:

$$\int_{\Omega} w c_t + M \int_{\Omega} \nabla w \cdot \nabla \mu = M \oint_{\partial\Omega} w (\nabla \mu \cdot \mathbf{n}) dS. \quad (12.1)$$

The entire influence of the boundary is that one surface integral. What you do with it *is* the boundary condition.

Natural (no-flux). Setting $\nabla \mu \cdot \mathbf{n} = 0$ makes the surface term vanish, and you impose it by *doing nothing* — simply omit the integral. This is why it is called *natural*: it is the condition the weak form adopts on its own. Physically it is a zero-flux wall: no mass crosses $\partial\Omega$, so total mass $\int_{\Omega} c dV$ is conserved to solver tolerance. Every spinodal run in Physics Chapter 2 is a no-flux run, which is why its mass drift was machine epsilon.

Dirichlet (fixed value). To hold the boundary composition at a prescribed g we cannot use the surface term — we *replace* the boundary rows of the assembled system with the identity, $A_{ii} \leftarrow 1$, right-hand side $\leftarrow g - x_i$ (the Newton-increment form). This pins (c, μ) at those nodes exactly. It is the treatment the manufactured-solution study of Chapter 11 needed (the exact field is not no-flux), and it models a composition held fixed at a contact. Crucially, a pinned boundary *is* a reservoir: whatever flux is needed to maintain the value flows freely, so mass is **not** conserved — by design.

A boundary condition changes what you conserve. No-flux conserves mass because it forbids boundary flux. Dirichlet abandons conservation because it fixes a value and lets flux adjust. Neither is “more correct” — they model different physical situations (a sealed domain vs. a contact held at a set composition). Choosing wrong gives a plausible-looking but physically wrong answer.

A third kind: wall free energy. A real thin film often *wets* its substrate: one component prefers the wall. That is a surface free energy $\oint g_s(c) dS$ added to (2.1), which makes the natural flux *nonzero* (a Robin-type wetting condition) rather than pinning a value. It is genuine physics, but it lives in the multiphase film brick, not this binary Cahn–Hilliard brick (see docs/dev/2026-07-13-m5-device-assembly.md, the A2 wall term); we name it here for completeness and use it in the device track. This tutorial measures the two treatments the binary brick implements.

12.2 Run it

Run it.

```
cd computational/02_boundary_conditions
python run.py                # both boundary treatments, compared
```

It marches the *identical* spinodal blend (same fixed seed) twice — once no-flux, once with every box edge pinned to $c = 0.9$ — and prints the mass drift and boundary layer of each. Compare with EXPECTED.md and Fig. 12.1–12.2.

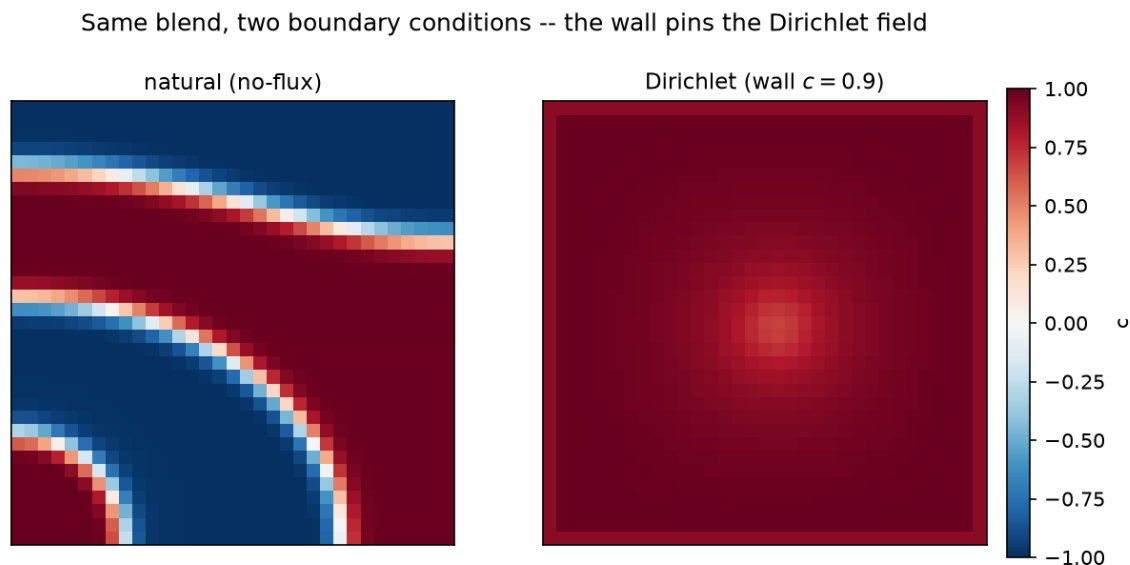


Figure 12.1: The same blend under the two boundary conditions. Left: no-flux — domains meet the wall freely. Right: Dirichlet — the edge is pinned to $c = 0.9$ and that phase is drawn to the boundary, reorganizing the morphology. The measured field difference is 2.00 (the full order-parameter range).

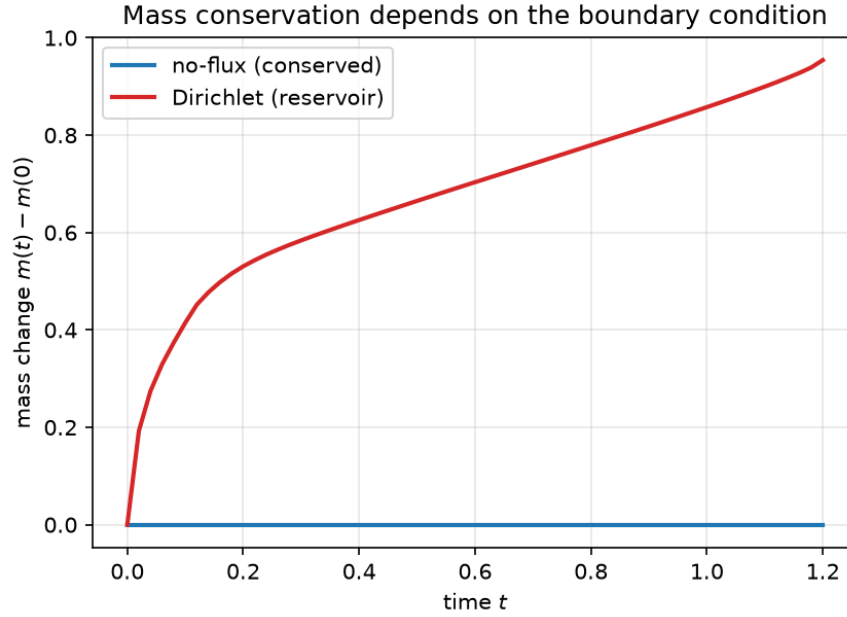


Figure 12.2: Mass change $m(t) - m(0)$. The no-flux wall conserves mass to machine precision (flat at zero); the Dirichlet wall acts as a reservoir and draws material in ($\Delta m \approx 0.95$).

Measured self-check (fixed seed in `bc.py`; field values are seed-dependent, the conservation/pinning contrast is not).

	natural (no-flux)	Dirichlet ($c = 0.9$)
mass drift $ \Delta m $	1.3×10^{-16}	0.95
edge composition	-0.13 (free)	+0.90 (pinned)
field difference $\max c_{\text{nf}} - c_{\text{dir}} $		2.00

12.3 Code walk-through

The comparison lives in `bc.py`, driving the same production brick.

Selecting the boundary.

```
# natural: pass nothing -- the default weak form is no-flux
st = CahnHilliardStepper(dm, M, kap, dt, order=1)

# Dirichlet: pass the boundary node indices + the data functions
st = CahnHilliardStepper(dm, M, kap, dt, order=1, dirichlet=bidx,
                        gc_fn=lambda x,t: wall, gm_fn=lambda x,t: 0)
```

`boundary_free_nodes` finds the box-edge nodes; `gc_fn`, `gm_fn` supply the pinned (c, μ) values. The natural case needs no argument at all — the load-bearing point of the concept.

Measuring the effect.

```
m0 = mass(st, st.hist[0])
for _ in range(steps): c, _ = st.step()
drift = abs(mass(st, c) - m0) # ~1e-16 no-flux; ~0.95 Dirichlet
```

`mass` integrates $\int_{\Omega} c dV$ with the solver's own quadrature — the same conservation diagnostic as Physics P1. Running both modes from the identical seeded initial field isolates the boundary's effect in the field difference.

12.4 Explore on your own

Q12.4.1 Sweep the Dirichlet wall value $g \in \{-0.9, 0, +0.9\}$. Predict the sign and rough size of Δm for each before running, then check. For which g is mass *approximately* conserved, and is that a coincidence of the initial mean?

Q12.4.2 Pin only *two* opposite edges (left/right) instead of all four, leaving top/bottom no-flux. How does the boundary layer and the mass drift change? This mixed condition is common in device geometries (contacts on two sides, sealed on the others).

Q12.4.3 The no-flux mass drift is $\sim 10^{-16}$ — machine epsilon, not solver tolerance. Refine the mesh one level and re-measure: does the drift grow with the number of nodes (accumulated round-off) or stay at machine level? What does that tell you about how conservation is enforced here?

Q12.4.4 Under Dirichlet, the boundary μ is pinned to 0. Is that physical? Argue what μ at a composition-pinned contact *should* be, and why pinning both c and μ (as this brick does) is a modelling choice, not a law. What would a c -only Dirichlet require of the solver?

Q12.4.5 Estimate the “cost” of each boundary condition: count how many rows the Dirichlet case replaces versus the interior, and argue why the natural condition is not only physically distinct but computationally free.

Chapter 13

Spatial and temporal adaptivity

Key idea. Resolution should follow the physics. In space, *octree refinement* puts small elements only where the field is sharp (the interfaces) and leaves the bulk coarse. In time, an *LTE-controlled* step ladder shrinks Δt during the violent onset and grows it through slow coarsening — but growing Δt safely forces BDF2 to carry its *variable-coefficient* form, or its order silently collapses.

This concept measures both adaptivities on the Cahn–Hilliard brick. It builds directly on the convergence study (Chapter 11: the BDF2 order that adaptivity must preserve) and enables the long phase-field horizons the physics track needs.

13.1 Spatial adaptivity: octree refinement

A spinodal field is sharp at its interfaces and smooth everywhere else. A uniform mesh fine enough to resolve the interfaces wastes almost all its elements on the flat bulk. An *octree* mesh refines adaptively: starting from one cell, it subdivides only where a criterion says to. Here we refine any element within a band of a circular interface and leave the rest coarse.

Hanging nodes. When a fine element meets a coarse one, the fine element has a node in the middle of the coarse element’s edge — a *hanging node* that is not a true degree of freedom. It is *constrained* to the average of its coarse neighbors, and that constraint lives in the matrix T (recall $A = T^\top K T$). The solver never sees the hanging nodes as unknowns; `build_constraints` builds T , and the *same* stepper runs unchanged — which is why the tutorial can just hand the CH brick an adaptive mesh and step.

The measured result: refining along one circle at level 6 gives 1,173 nodes against 4,225 for the uniform mesh at the same finest h — a $3.6\times$ reduction in degrees of freedom (Fig. 13.1), and the CH brick steps on it. In 3-D the savings compound (a surface is a vanishing fraction of a volume), which is what makes device-scale films tractable at all (Chapter 15).

13.2 Temporal adaptivity: the LTE step ladder

Phase separation is violent at onset and glacial during coarsening — the natural time scale spans orders of magnitude. A fixed Δt must be small enough for the onset, wasting effort forever after. The *adaptive* march sizes each step to the physics using a *local truncation error* (LTE) estimate:

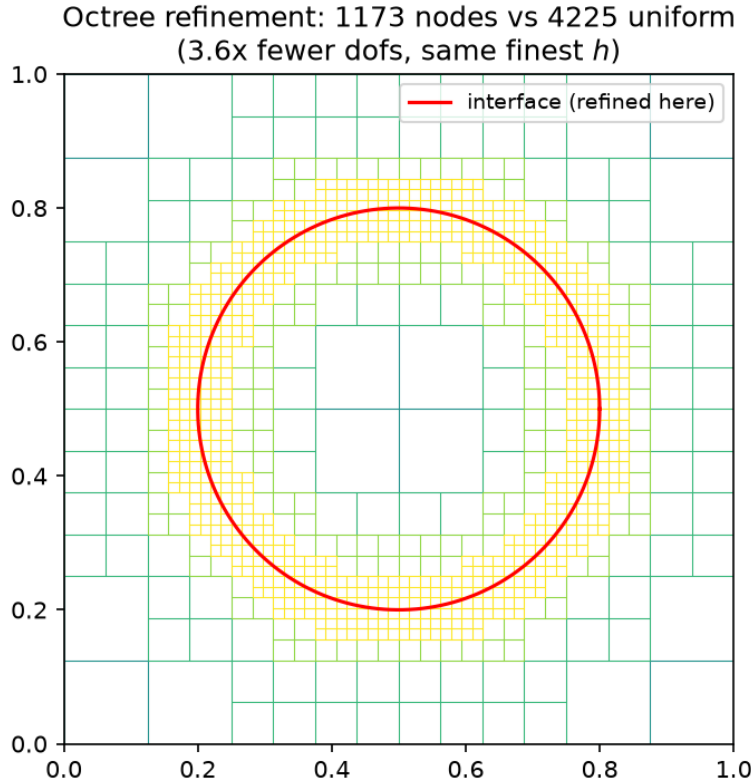


Figure 13.1: Octree refinement along a circular interface: small elements hug the interface (red), large elements fill the bulk. The adaptive mesh has 1,173 nodes versus 4,225 for a uniform mesh at the same finest resolution ($3.6\times$ fewer dofs). Element color is refinement level.

it takes one step of size Δt and two of size $\Delta t/2$ from the same state, and the difference (scaled by $2^q - 1$) estimates the error of the step. If it is below tolerance, accept and *grow* Δt ; otherwise reject, halve, and retry.

Run it.

```
cd computational/03_adaptivity
python run.py                # octree + time-ladder + BDF2 order
```

Compare the printed studies with EXPECTED.md and Fig. 13.1–13.2.

The measured ladder over a quench to $t = 0.8$: Δt collapses to its floor 1.0×10^{-5} during onset, then climbs to 6.6×10^{-2} during coarsening — a range of $6623\times$ (Fig. 13.2). It reaches $t = 0.8$ in 142 steps; a fixed march safe throughout would need the onset step size the whole way — about 80,000 steps, a $563\times$ saving.

13.3 Why variable-coefficient BDF2

Growing Δt safely has a hidden cost. The textbook BDF2 coefficients $\frac{3}{2}, [2, -\frac{1}{2}]$ are derived assuming a *constant* step. Feed them a step that changed from the last one and the scheme is only *formally* second order — its *measured* order collapses toward 1. The retrofit measured exactly this:

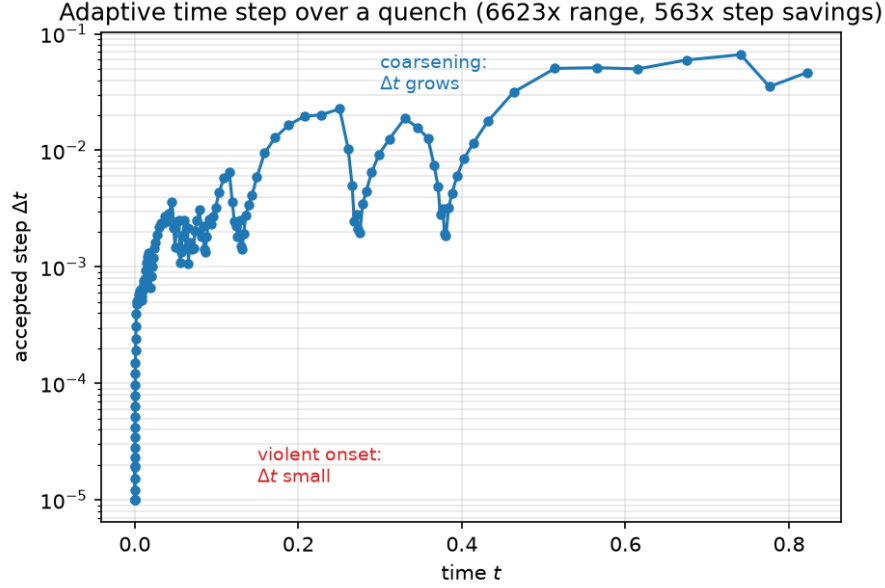


Figure 13.2: The accepted time step over a quench (log scale). Δt is tiny during the spinodal onset and grows more than four orders of magnitude through coarsening. 142 adaptive steps replace $\sim 80,000$ fixed steps ($563\times$).

on an alternating $(\Delta t, \Delta t/2)$ sequence the constant-coefficient form converged at order 0.90/0.95 (audit doc, G3). The fix is to rebuild the coefficients each step from the actual ratio $r = \Delta t / \Delta t_{\text{prev}}$:

$$c_0 = \frac{1 + 2r}{1 + r}, \quad \text{history weights } \left[1 + r, -\frac{r^2}{1+r} \right], \quad (13.1)$$

which reduces to the constant-step form at $r = 1$ (bit-exactly, so fixed- Δt runs are unchanged) and stays consistent when $r \neq 1$. The tutorial re-measures it on the same alternating sequence: order 2.01 — second order restored.

This is why C1 and C3 are one story. Adaptivity is worthless if it silently destroys the accuracy you measured in Chapter 11. The LTE controller and the variable-coefficient scheme are a matched pair: the controller *varies* Δt ; the variable-coefficient BDF2 is what keeps the order-2 accuracy while it does.

Measured self-check (fixed seed in `adaptivity.py`; node counts are exact, step counts are seed/card-dependent).

study	measured	note
octree nodes (adaptive / uniform)	1,173 / 4,225	$3.6\times$ fewer
adaptive steps to $t = 0.8$	142	vs $\sim 80,000$ fixed
Δt range	$1.0 \times 10^{-5} \rightarrow 6.6 \times 10^{-2}$	$6623\times$
step savings	$563\times$	
variable-coeff BDF2 order	2.01	vs 0.90/0.95 const-coeff

13.4 Code walk-through

Octree from a refinement predicate.

```
def refine_fn(centers, h):          # True where to subdivide
    r = |centers - 0.5|
    return abs(r - 0.30) < 1.5*h    # a band around the circle
atree = build_adaptive(refine_fn, max_level=6, dim=2)
```

`build_adaptive` subdivides until no flagged element can refine further; `build_constraints` then ties the hanging nodes.

The LTE-controlled march.

```
ts, dts = adaptive_march(st, t_end=0.8, tol=5e-4)    # returns the ladder
```

Inside, each step is step-doubled; accepted steps update Δt by $\text{safety} \cdot (\text{tol}/\text{LTE})^{1/(q+1)}$ (clamped). Rejects rewind the state *and* Δt_{prev} — so the BDF2 history spacing is never corrupted by a rejected step.

Measuring the variable-dt order.

```
ref = fixed_march(dt=2e-4)          # fine reference
errs = [max|var_march((dt, dt/2)) - ref| for dt in (8e-3, 4e-3, 2e-3)]
order = mean(log2(errs[i]/errs[i+1]))    # ~2.0, not ~1.0
```

The alternating $(\Delta t, \Delta t/2)$ sequence forces a fresh ratio r on every step — the hardest test of the variable-coefficient form.

13.5 Explore on your own

Q13.5.1 Tighten the LTE tolerance from 5×10^{-4} to 10^{-4} . How do the step count and the peak Δt change? Verify that the accuracy improves roughly in proportion to the tolerance (the controller is doing its job).

Q13.5.2 Widen the octree refinement band from $1.5h$ to $3h$. How many more nodes does it add, and does the CH step’s cost track the node count or the element count? Which is the right “size” of an adaptive mesh?

Q13.5.3 Refine along *two* circles instead of one. Predict the node count before running (a surface scales as its length here), then check. How would this scaling differ in 3-D?

Q13.5.4 Force the constant-coefficient bug back: run the alternating sequence but with a *fixed* Δt (so $r \equiv 1$) and confirm you recover order 2 — then argue why the alternating sequence, not the fixed one, is the test that exposes the variable-coefficient issue.

Q13.5.5 The onset drives Δt to its floor. Lower the floor `dt_min` by $10\times$ and see whether the onset resolves better or just wastes steps. What physically sets the smallest *useful* Δt during spinodal onset?

Chapter 14

The solver ecosystem

Key idea. There is no single best linear solver. A direct factorization (cuDSS) is unbeatable in 2-D and up to a memory wall in 3-D; past that wall it dies, and a block preconditioner (`blockch`) wins by an order of magnitude; past the point where even the matrix will not fit, only a matrix-free method survives. Choosing the solver is choosing where on that curve your problem sits.

This concept teaches how to choose, using a small live 2-D comparison plus the measured 3-D scaling tables from the device-assembly and `blockch` development notes.

14.1 Three families of solver

Each Newton iterate solves $A\delta = r$, where A is the mixed (c, μ) Cahn–Hilliard Jacobian — a *saddle*-structured sparse matrix. Three families compete:

- **Direct** (LU factorization: `splu` on the CPU, **cuDSS** on the GPU). Robust, tuning-free, exact to round-off. But the triangular factors *fill in*: their nonzeros grow far faster than A 's, cheaply in 2-D and punishingly in 3-D, until the factors do not fit in memory.
- **Block-preconditioned iterative** (`blockch` / `blockch_dev`). An outer FGMRES wrapped around a block factorization that *never* forms the global LU. It is latency-bound at small n (loses badly in 2-D), but its memory and per-solve cost scale gently — so it wins in 3-D past the point cuDSS runs out of memory.
- **Matrix-free**. Never stores A at all — applies the action $A\mathbf{v}$ batch-wise through the element kernels. The only survivor once even the sparse A overflows 32-bit indexing or the card's memory.

Why the choice flips with dimension. In 2-D the interface is a curve and the matrix bandwidth is modest, so direct fill-in is cheap — cuDSS dominates. In 3-D the interface is a surface, the bandwidth explodes, and the LU factors blow past the card's memory; that is exactly where the block preconditioner takes over, and beyond *it*, matrix-free. Solver choice is a function of where on the cost curve your problem sits.

14.2 The 2-D crossover, measured live

We assemble the *real* Cahn–Hilliard Jacobian at three mesh sizes and time the linear solve (factorize + solve) with CPU-direct (`splu`) and GPU-direct (`cuDSS`). Isolating the solve matters: the full step is host-assembly-bound in this brick, which would swamp the solver signal.

Run it.

```
cd computational/04_solvers
python run.py                # live 2-D benchmark + cited 3-D + the rule
```

At 2,178 dofs, `cuDSS` is $\sim 0.1\times$ the speed of `splu` — i.e. *slower*: the GPU’s launch and transfer overhead dominates a tiny solve. By 33,282 dofs `cuDSS` is $\sim 2.1\times$ *faster* and pulling away. The crossover sits near 10^4 dofs (Fig. 14.1). The block preconditioner is 26–500 $\times$ slower than `cuDSS` across this whole 2-D range (dev note, cited) — you would never use it here.

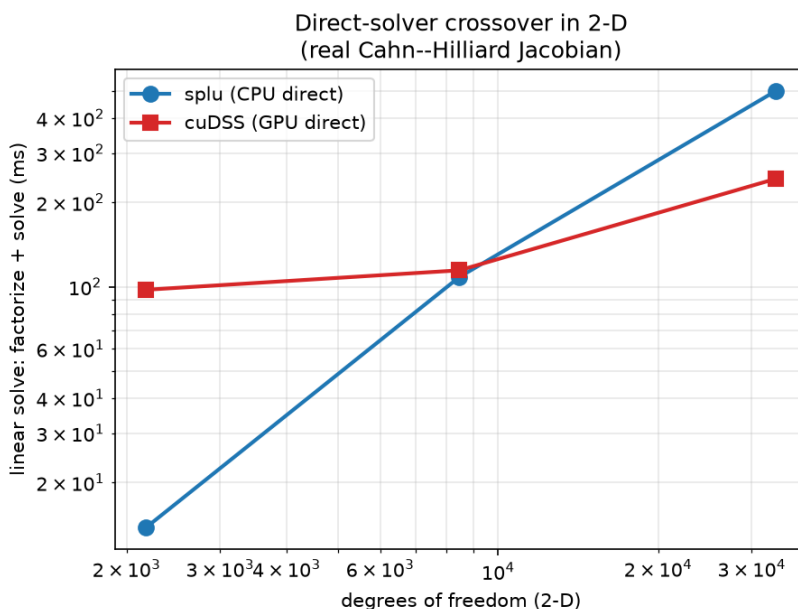


Figure 14.1: Direct-solver crossover in 2-D on the real Cahn–Hilliard Jacobian: CPU `splu` wins at small sizes (GPU overhead dominates), GPU `cuDSS` wins as the problem grows and keeps scaling better. Timings vary run-to-run; the crossover shape does not.

14.3 The 3-D story, measured (and cited)

The 3-D numbers take minutes per step to produce, so we cite the development notes’ measured tables rather than re-run them. The picture is decisive (Fig. 14.2):

- `cuDSS`’s factorization fill hits the 48 GB card at $\sim 811,008$ dofs — beyond that the factorization crawls into hybrid memory and never returns.
- `blockch_dev` beats `cuDSS` $8.4\times$ on the slab64 solve (2.10 vs 17.7 s/call) *and* keeps marching at 811,008 dofs, where `cuDSS` ceilings.

- a *block-masked* cuDSS pattern cuts the 3-D fill ($17.7 \rightarrow 3.70$ s/call) — a strong option under $\sim 5 \times 10^5$ dofs, but the memory wall above it is unchanged in kind.
- **AMGX verdict: no.** At production Δt the inner solves are mass-dominated (well-conditioned), so algebraic multigrid has no $O(h^{-1})$ growth to flatten and adds nothing.
- matrix-free (and `blockch`) push the ceiling to 6,389,760 dofs ($128^2 \times 64$) on a single 48 GB card — a size cuDSS cannot touch on any card here.

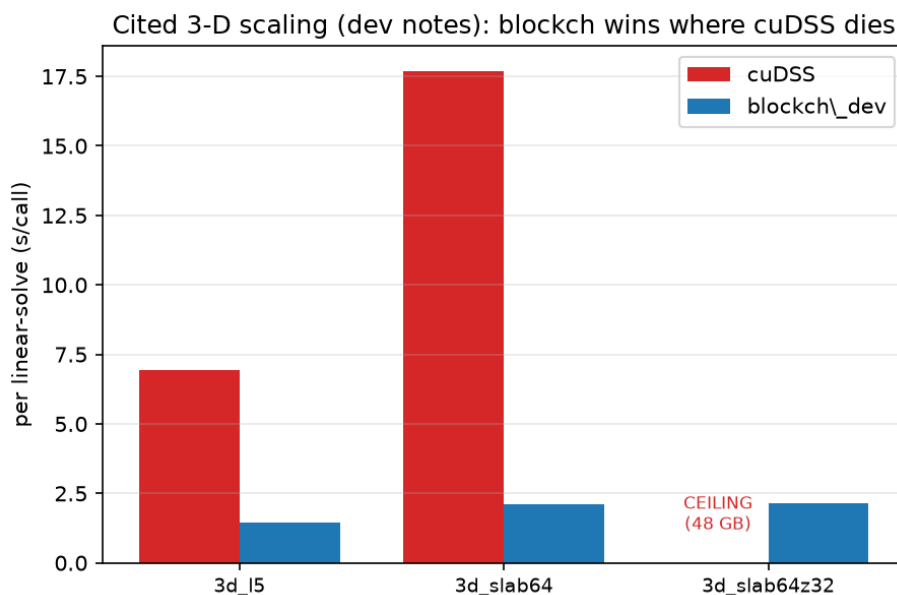


Figure 14.2: Cited 3-D per-solve cost (dev notes): cuDSS versus `blockch_dev`. `blockch` is 8.4× cheaper at slab64 and keeps marching at the size where cuDSS’s factorization ceilings on a 48 GB card.

The decision, as a rule (`choose_solver` in the tutorial):

regime	dofs (order)	solver
2-D, small	$< 10^4$	<code>splu</code> (CPU direct)
2-D, large	$> 10^4$	cuDSS (GPU direct)
3-D, small	$< 5 \times 10^5$	cuDSS / masked cuDSS
3-D, large	up to $\sim 1.5 \times 10^6$	blockch_dev
3-D, extreme	$> \text{int32}$ / memory	matrix-free (or multi-GPU)

14.4 Code walk-through

Capturing the real operator.

```
A, b, dofs, nnz = capture_ch_system(level)    # intercepts the solve
```

The benchmark does not invent a matrix — it runs one real CH Newton iterate and intercepts the stepper’s linear solve to grab the exact saddle Jacobian A and residual b .

The fair timing.

```
t_splu = time(lambda: splu(A.tocsc()).solve(b))    # CPU, refactor
```

```
t_cudss = time(lambda: solve_linear(A, b, solver="cudss", cache={})) # GPU
```

Both refactor on every call (fresh `cache={}()`), so the comparison includes factorization — the honest cost of a direct solve inside a Newton loop where A changes each iterate.

14.5 Explore on your own

Q14.5.1 Extend the live 2-D sweep to level 8 ($\sim 1.3 \times 10^5$ dofs). Does cuDSS's lead over `splu` keep growing, and does the solve time scale closer to $O(n)$ or $O(n^{1.5})$? What does the observed exponent say about 2-D direct fill-in?

Q14.5.2 Time *just the solve* versus *factorize + solve* by reusing one factorization for many right-hand sides. In a Newton loop A changes every iterate, so you cannot reuse — but where in this simulator *could* you (hint: a fixed-point or a frozen-Jacobian scheme), and how much would it save?

Q14.5.3 The dev note reports `blockch_dev` at $8.4\times$ cuDSS on the slab64 *solve* but only $2.6\times$ on the *step*. What does the rest of the step cost, and why does the solver speedup not carry through one-to-one to wall-clock? (Look at the assembly and outer-FGMRES costs.)

Q14.5.4 Using the cited fill behavior, estimate the dofs at which cuDSS would ceiling on an 80 GB A100. Then decide: for a $96 \times 96 \times 48$ film, which solver, and would masked cuDSS change your answer?

Q14.5.5 AMGX was measured to add nothing because the inners are mass-dominated at production Δt . Construct the one regime where that argument breaks (hint: what happens to the conditioning when the adaptive ladder collapses Δt during a deep quench?), and say what you would measure to confirm it.

Chapter 15

Two dimensions versus three

Key idea. Going from 2-D to 3-D is not a small constant factor. Every level of refinement multiplies the degrees of freedom by 4 in 2-D but 8 in 3-D, the stencil thickens from 9 to 27 neighbors, the matrix nonzeros can overflow 32-bit indexing, and the direct-solver fill explodes — which is precisely why 3-D forces the device assembly path and the `blockch` solver of Chapter 14.

This concept measures the 2-D-vs-3-D growth live on small meshes and connects it to the device-scale ladders from the dev notes. It is the capstone of the computational track: it explains *why* everything in Chapter 14 (the device assembly path, the `blockch` solver, the matrix-free frontier) exists.

15.1 Three things grow at once

Refine a mesh by one level and the number of elements multiplies by 2^{dim} — 4 in 2-D, 8 in 3-D. That is the obvious growth. Two less-obvious ones compound it:

- **The stencil thickens.** A node couples to its $3^{\text{dim}} - 1$ neighbors: 8 in 2-D (a 9-point stencil), 26 in 3-D (a 27-point stencil). For the mixed (c, μ) system that is ~ 18 nonzeros per dof in 2-D and ~ 47 in 3-D (measured below). The matrix is not just bigger — it is *denser per row*.
- **The direct-solver fill explodes.** A larger 3-D bandwidth makes the LU factors fill in far worse than in 2-D — the cuDSS memory ceiling of Chapter 14.

The compounding. 2-D dofs grow $\times 4/\text{level}$ and carry ~ 18 nnz/dof; 3-D dofs grow $\times 8/\text{level}$ and carry ~ 50 nnz/dof. Four levels of refinement is $256\times$ the dofs in 2-D but $4096\times$ in 3-D — and each 3-D dof is $\sim 3\times$ denser. This is why a code that is instant in 2-D needs a GPU, a device assembler, and a scalable solver in 3-D.

15.2 Run it

Run it.

```
cd computational/05_two_vs_three
python run.py                # live 2-D vs tiny-3-D + cited ladder (~30-40 s)
```

The 3-D level-5 step is a real 71,874-dof Cahn–Hilliard step on the box (a few seconds).

Compare with `EXPECTED.md` and Fig. 15.1–15.2.

Measured: dofs grow $\times 3.9/\text{level}$ in 2-D and $\times 7.0/\text{level}$ in 3-D (approaching the theoretical 4 and 8; Fig. 15.1). At comparable size — 33,282 dofs in 2-D versus 71,874 in 3-D — the 3-D matrix has $6.2\times$ the nonzeros and the step takes $6.8\times$ as long.

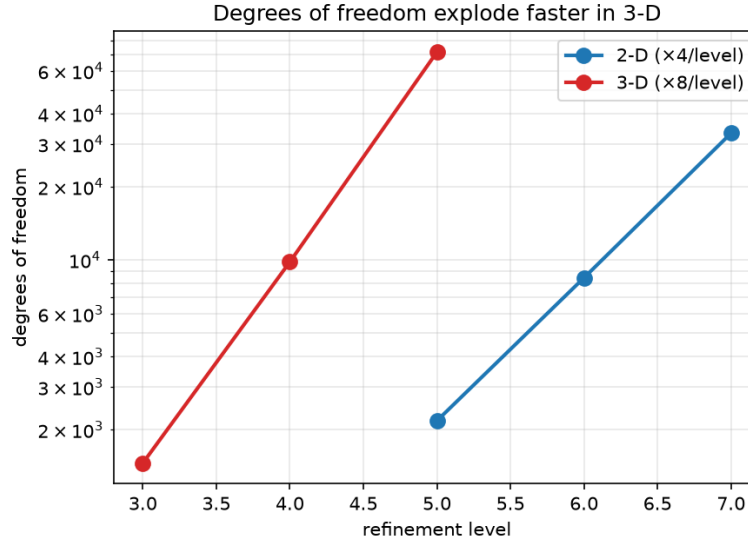


Figure 15.1: Degrees of freedom versus refinement level (log scale). Each 2-D level multiplies dofs by ~ 4 ; each 3-D level by ~ 8 . The gap widens without bound.

15.3 The device-scale ladder

The live meshes stop at tens of thousands of dofs; production 3-D films run at *millions*. Figure 15.2 places the cited device-scale points (dev notes) on the same axes as the live runs, with the 32-bit index ceiling drawn in.

- `3d_slab64` (417,792 dofs, 65M nnz) is the largest cuDSS-stable case on a 48 GB card — 3% of the int32 ceiling.
- `3d_film128` (6,389,760 dofs, 1.02B nnz) runs on *one* 48 GB card with `blockch`; cuDSS cannot.
- at $(M=3, K=2)$ production physics, `128x128x48` (8,028,160 dofs) *overflows* the 32-bit nnz arithmetic — which is exactly why the block-masked pattern exists (it drops the structural zeros the superset stores).
- `256x256x128` (84,541,440 dofs, 22.8B nnz) has no single-card stored CSR at all — only matrix-free or multi-GPU.

Measured self-check (counts are exact; step times are card-dependent).

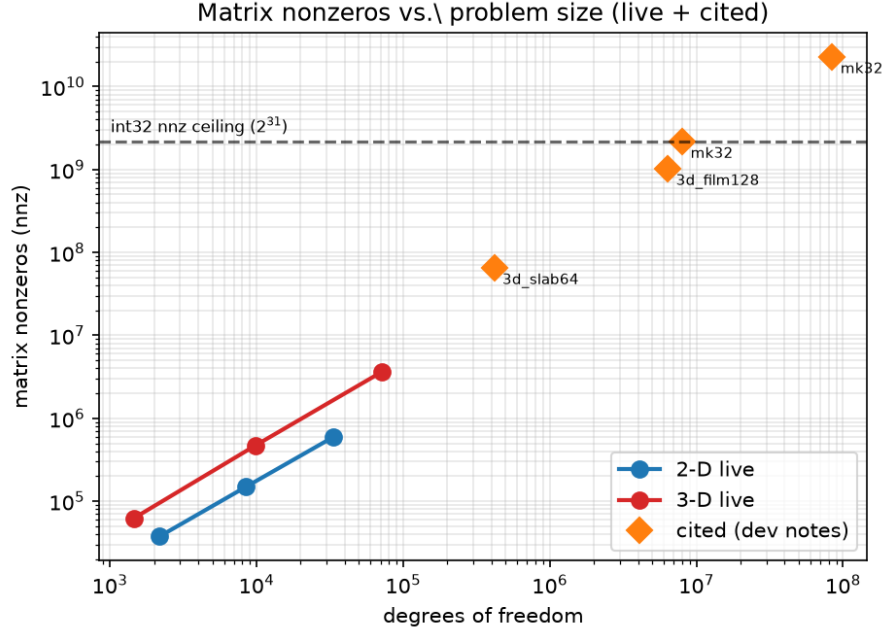


Figure 15.2: Matrix nonzeros versus problem size: live 2-D/3-D runs plus the cited device-scale points. The dashed line is the 32-bit CSR index ceiling (2^{31}); the $(M=3, K=2)$ 128-class films cross it, forcing the block-masked pattern, and the Nova-class case (22.8 B nnz) has no stored-CSR form at all.

	2-D	3-D
dofs per level	$\times 3.9$	$\times 7.0$
nnz per dof (stencil)	18	47
at comparable size (33,282 vs 71,874 dofs):		
nnz ratio (3-D / 2-D)	6.2×	
step-time ratio	6.8×	

15.4 Code walk-through

Measuring a real system's size.

```
dofs, nnz, step_s = _capture_nnz(dim, level)  # one real CH step
row = scaling_row(dim, level)                # + nnz/dof, mem estimate
```

As in Chapter 14, the benchmark assembles the *real* Cahn–Hilliard Jacobian and reads its nnz — the true storage the solver faces, not a formula.

The int32 ceiling.

```
INT32_MAX = 2**31
int32_headroom(nnz)  # nnz / 2^31: >1 means the CSR overflows
```

The CSR index arithmetic is 32-bit; once nnz passes 2^{31} the slot arithmetic overflows, and the code must either mask the pattern down or go matrix-free.

15.5 Explore on your own

Q15.5.1 Fit a power law to the live 3-D step times ($\text{step} \sim \text{dofs}^\alpha$). Is α closer to 1 (ideal), 1.5, or worse? Which stage — host assembly or the solve — drives it, and does that match the C4 story of where 3-D cost lives?

Q15.5.2 Compute the crossover level at which a 3-D mesh overtakes a 2-D mesh of the same refinement level in *total nnz*. Then compute the level at which a uniform 3-D mesh would overflow int32 at ($M=1$) binary physics — and compare with the cited ($M=3, K=2$) overflow at $128 \times 128 \times 48$.

Q15.5.3 The block-masked pattern drops structural zeros to fit int32. Using the cited ($M=3, K=2$) mask density (54/100 live blocks), predict the masked nnz for $128 \times 128 \times 48$ and check it against the dev note’s 1.16B. At what mesh does even the *masked* pattern overflow?

Q15.5.4 Estimate the host memory to store the 3-D level-6 CH matrix ($\sim 5.7 \times 10^5$ dofs) from the measured nnz/dof, and compare with the GPU memory a direct factorization would need (use the C4 fill behavior). Which runs out first, and on which card?

Q15.5.5 An octree mesh (Chapter 13) refined only at the interfaces changes the 3-D growth law: a surface scales as area, not volume. Estimate how many dofs an interface-refined 3-D film would have versus the uniform 71,874, and argue why adaptivity matters far more in 3-D than in 2-D.

Part III

Differentiation

Chapter 16

What differentiable simulation is

Key idea. A classical solver answers “*what happens?*”. A differentiable solver also answers “*what should I change?*” — it returns the gradient of any scalar output J with respect to any input parameter p (material coefficients, a process schedule, geometry). That single object dJ/dp turns the simulator from a *predictor* into an *instrument for design and inference*. And because a gradient you cannot trust is worse than none, DiffSim computes it *three independent ways* and checks that they agree.

16.1 Why a gradient

Every question we ask of a morphology simulator downstream — fit this material parameter to a measured morphology, choose a quench schedule that hits a target crystallinity, learn a free-energy functional from snapshots — is an *optimization*. Optimization needs a gradient: dJ/dp says how a scalar objective J (a domain size, an interfacial area, a crystalline fraction) moves when you nudge a parameter p .

The naive way to get dJ/dp is finite differences: run the simulation at $p+\epsilon$ and $p-\epsilon$ and subtract. That costs *two forward solves per parameter* and forces you to tune ϵ (too big: truncation error; too small: roundoff). With hundreds of parameters — a temperature schedule sampled every step, a free energy with many coefficients — it is hopeless. The **adjoint method** returns dJ/dp for *all* parameters at once, at the cost of a single extra (linear) solve backward through the trajectory, exact to machine precision. That $O(1)$ -in-parameters cost is what makes design possible.

16.2 The three ways to a gradient

DiffSim’s differentiable core (`src/diffsim/adjoint/`) computes the same gradient three ways, and the tutorial runs all three live:

1. **Adjoint** — a hand-derived reverse sweep. We do *not* tape the Newton iterations; we differentiate only the converged discrete relation $R_n(x_n; x_{\text{hist}}, p) = 0$ (the implicit-function theorem). For $J = \sum_n j(x_n)$,

$$J_n^\top \lambda_n = \frac{\partial j}{\partial x_n} - \sum_{k \geq 1} \left(\frac{\partial R_{n+k}}{\partial x_n} \right)^\top \lambda_{n+k}, \quad \frac{\partial J}{\partial p} = - \sum_n \lambda_n^\top \frac{\partial R_n}{\partial p}, \quad (16.1)$$

where $J_n = \partial R_n / \partial x_n$ is *exactly* the Jacobian the forward Newton already assembled at convergence, and $\partial R_n / \partial p$ is analytic. This is the production capability.

2. **Autograd twin** — a faithful PyTorch reimplementation of the *same* discrete solver, differentiated by `torch.autograd` through the converged Newton loop. Different code, a different differentiation engine, the same math: an *independent* reference.
3. **Finite differences** — central differences $(J(p + \epsilon) - J(p - \epsilon))/2\epsilon$. No calculus at all; the ground truth every gradient must match, down to the FD error floor.

The adjoint differentiates the *real* brick. The discrete forward in `adjoint/phasefield.py` is a vectorised re-implementation of the production Cahn–Hilliard residual and Jacobian that is *bit-parity* with `CahnHilliardStepper` (measured $\max |\Delta c| \sim 10^{-16}$). So the gradient is taken through the same operator the research code marches — not a look-alike toy.

16.3 Run it

Run it.

```
cd differentiable/01_three_way_gradient
python run.py # 9x9 nodes, 3 steps, three-way check
```

It marches a short Flory–Huggins Cahn–Hilliard rollout, forms the scalar objective $J = \frac{1}{2}\|c_N - \frac{1}{2}\|^2$, and computes $dJ/d\{M, \kappa, \chi, A\}$ all three ways. It prints the table and **self-checks** that the methods agree — that agreement is the gate. Compare with `EXPECTED.md`.

Measured self-check (three-way gradient of $J = \frac{1}{2}\|c_N - \frac{1}{2}\|^2$ through a 3-step march, 9×9 nodes).

parameter	adjoint dJ/dp
M (mobility)	3.077×10^{-1}
κ (gradient)	-7.357×10^0
$\chi = B$ (Flory)	8.196×10^{-1}
A (entropic)	-1.668×10^0

Adjoint and autograd twin agree to $\sim$ machine precision (worst $\leq 1.8 \times 10^{-16}$): the *same* discrete math, two independent differentiation engines. Adjoint and finite differences agree to worst $\leq 1.5 \times 10^{-9}$: the FD truncation/roundoff floor.

16.4 Code walk-through

The tutorial’s core is `three_way.py`; it drives `src/diffsim/adjoint/phasefield.py`.

1. One reverse sweep, all gradients.

```
fwd = CHForward(dm, FHEnergy(A, B), M=M, kappa=kappa, dt=dt, order=order)
fwd.set_initial(ic); fwd.run(n_steps) # record the trajectory
dJdx = [zeros] * n_steps
dJdx[-1][0::2] = fwd.steps[-1]["c"] - target # dj/dc on the final
step
g_adj = CHAdjoint(fwd).gradient(dJdx, ["M", "kappa", "B", "A"])
```

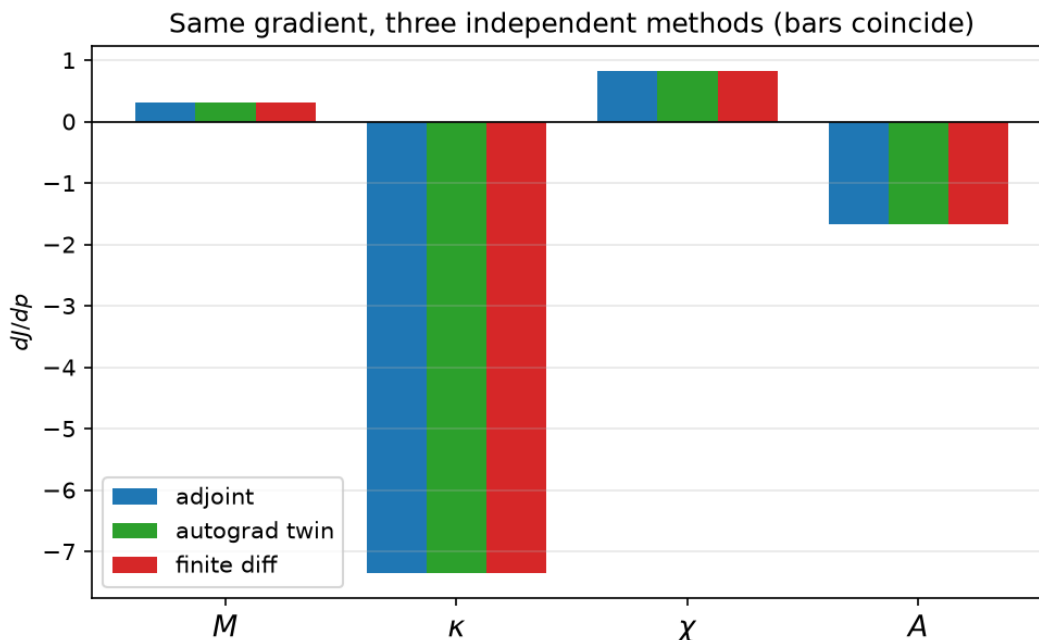


Figure 16.1: The same gradient, three ways. For each parameter the adjoint, autograd-twin, and finite-difference bars are *visually coincident* — that overlap is the message: three independent methods return one number. The signs are physical (raising χ drives the field away from the uniform $c = \frac{1}{2}$, so J grows; $dJ/d\chi > 0$).

gradient runs (16.1) backward through the recorded steps and returns *every* dJ/dp from that one sweep.

2. The independent twin.

```
twin = CHTwin(dm, energy="fh", dt=dt, order=order, device="cpu")
out = twin.march(c0, None, M, kappa, dict(A=A, B=B), n_steps) # leaf
      tensors
loss = 0.5 * ((out[-1][0] - target)**2).sum(); loss.backward()
```

Autograd differentiates through the converged Newton iterations; because the loop is driven to a tight residual, that returns the same implicit-function-theorem gradient the hand adjoint derives.

3. The ground truth. Central differences on each parameter — two forward solves apiece — with no calculus, is what pins the other two to reality.

16.5 Explore on your own

Q16.5.1 Switch to BDF2 (--order 2 --steps 4). The variable-coefficient history now couples *two* previous steps into each residual; does the three-way agreement survive? (It must — the adjoint's history cotangent carries both slots.)

Q16.5.2 Add a second objective, e.g. the interfacial energy $\frac{\kappa}{2} \|\nabla c_N\|^2$. Re-derive its seed dj/dc_N and confirm the adjoint still matches finite differences.

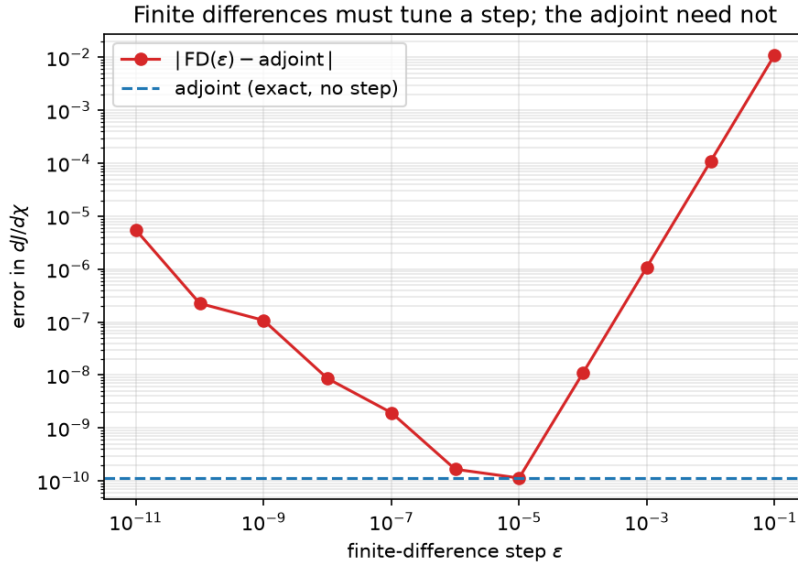


Figure 16.2: Why the adjoint, not finite differences. The error of the central difference for $dJ/d\chi$ is a V-shaped function of the step ϵ : on the right, $O(\epsilon^2)$ truncation error falls as ϵ shrinks; on the left, catastrophic cancellation (roundoff) takes over and it rises. The valley floor (best FD $\approx 1.1 \times 10^{-10}$) is the most FD can achieve — and it sits far above the adjoint, which is exact and needs no step to tune.

Q16.5.3 Time the adjoint vs. finite differences as you add parameters (say, a per-step schedule of length N). The adjoint cost is flat in N ; the FD cost grows linearly. At what N does the crossover make FD impractical?

Q16.5.4 Shrink the FD step ϵ toward 10^{-12} and watch the error in Fig. 16.2 *rise*. Explain why, and why the adjoint has no analogous floor.

Chapter 17

Morphology sensitivity

Key idea. With the gradient now trustworthy, ask a physical question: how sensitive is the *morphology* to the material parameters? Pick a scalar summary of the pattern — here the *demixing amplitude* $J = \frac{1}{2}\|c_N - \bar{c}\|^2$, the spatial variance of the composition — and the adjoint returns $dJ/d\chi$ and $dJ/d\kappa$ in one sweep. The *signs* are the physics: $dJ/d\chi > 0$ (interaction sharpens the pattern), $dJ/d\kappa < 0$ (gradient penalty blurs it).

17.1 A scalar handle on the morphology

A morphology is a whole field; to differentiate we need a scalar. Any morphology metric will do — domain size, interfacial area, a structure factor — and the adjoint machinery is identical: pick $J(c_N)$, hand it the seed dj/dc_N , get every dJ/dp . We use the demixing amplitude

$$J(c_N) = \frac{1}{2} \sum_i (c_{N,i} - \bar{c})^2, \quad \bar{c} = \text{mean}(c_0), \quad (17.1)$$

which is small for a nearly uniform blend and grows as the two phases separate — a scalar proxy for “how demixed is the film”. Because the Cahn–Hilliard dynamics conserve mass, $\bar{c}$ is fixed by the initial condition, so the seed $dj/dc_N = (c_N - \bar{c})$ is exact and clean.

The two sensitivities have opposite, and physical, signs. Above the spinodal ($\chi > 2A$) a larger interaction χ deepens the enthalpic well and drives demixing harder, so J grows: $dJ/d\chi > 0$. The gradient penalty $\frac{\kappa}{2}|\nabla c|^2$ instead *opposes* composition variation — a larger κ thickens interfaces and smooths the field — so $dJ/d\kappa < 0$. The adjoint recovers both, and finite differences confirm them.

17.2 Run it

Run it.

```
cd differentiable/02_morphology_sensitivity
python run.py # dJ/dchi and dJ/dkappa, FD-verified
```

It marches a short Flory–Huggins rollout at $\chi = 2.5$, $\kappa = 0.01$ (interior data, so the logs stay smooth), forms (17.1), and prints $dJ/d\chi$, $dJ/d\kappa$ with their finite-difference checks and sign assertions. Compare with `EXPECTED.md`.

Measured self-check (6-step march, $\chi = 2.5$, $\kappa = 0.01$).

	value	meaning
demixing amplitude J	3.095×10^{-1}	how separated the blend is
$dJ/d\chi$	7.433×10^{-1}	> 0 : interaction sharpens
$dJ/d\kappa$	-6.691×10^0	< 0 : gradient penalty blurs
worst adjoint-vs-FD	1.0×10^{-9}	the FD-verified gate

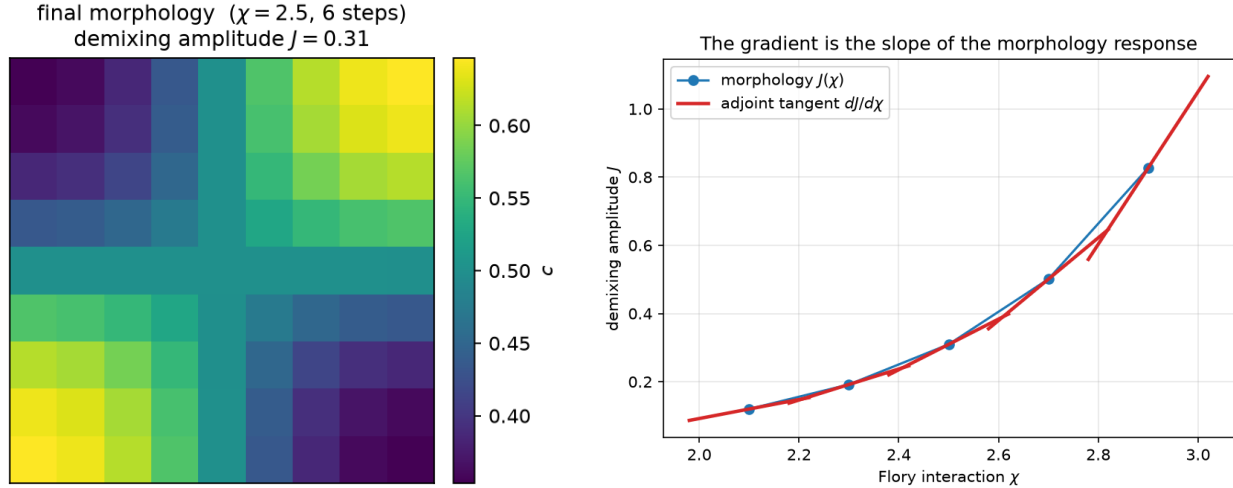


Figure 17.1: Left: the final morphology whose scalar summary J we differentiate. Right: the demixing amplitude $J(\chi)$ sampled across interaction strengths (blue), with the adjoint tangents $dJ/d\chi$ (red) drawn at each point. The tangents lie *along* the response curve — the gradient is the local slope of the morphology’s answer to a change in χ .

17.3 Code walk-through

The seed is the only thing that changes from D1; the adjoint sweep is identical.

```
fwd, cN = march(dm, coords, chi, kappa, n_steps) # short CH rollout
dJdx[-1][0::2] = cN - c_bar # dj/dc_N for eq. (D2)
g = CHAdjoint(fwd).gradient(dJdx, ["B", "kappa"]) # -> dJ/dchi, dJ/dkappa
```

Swap in any other differentiable morphology metric — redefine $J(c_N)$ and its seed — and the same call returns its sensitivities. That substitutability is the point: the adjoint does not care what scalar you put on top of the field.

17.4 Explore on your own

Q17.4.1 Replace the demixing amplitude with a genuine interfacial-area proxy, $J = \frac{1}{2}\|\nabla c_N\|^2$. Derive its seed and check the sign of $dJ/d\kappa$ — does a thicker interface raise or lower this metric?

Q17.4.2 Sweep χ from below the spinodal ($\chi < 2$) to well above. Where does $dJ/d\chi$ turn on, and does the onset line up with the spinodal threshold from Physics P1?

Q17.4.3 Push the rollout longer (`--steps 20`) and watch the Flory–Huggins logs approach the walls. At what point does the forward solve fail, and what does that tell you about differentiating stiff free energies near the pure phases?

Chapter 18

Recovering parameters from a morphology

Key idea. Turn the arrow around. Instead of “given the material, what morphology?”, ask “given the morphology, what material?”. This is an *inverse problem*: minimise the misfit between the simulated and the target field over the material parameters, using the adjoint gradient. It is the tutorial-scale version of *learning the material from the morphology* — the loop that, at research scale, fits a free-energy functional to measured micrographs.

18.1 The inverse problem

Given a target composition field c_N^{target} (from an experiment, or — here — from a known ground-truth material so we can check the answer), find the parameters $\theta = (\chi, \kappa)$ that reproduce it. The objective is the field misfit

$$L(\theta) = \frac{1}{2} \|c_N(\theta) - c_N^{\text{target}}\|^2, \quad (18.1)$$

and we minimise it by quasi-Newton descent (L-BFGS-B). Every optimiser step needs L and its gradient $dL/d\theta$; the adjoint delivers both from one forward rollout plus one reverse sweep — the seed is $dj/dc_N = c_N - c_N^{\text{target}}$, exactly the D2 machinery with a different scalar on top.

Identifiability. The recovery works because χ and κ leave *distinguishable* fingerprints: the interaction sets *how much* the blend demixes, the gradient penalty sets the *interface width*. When two parameters move the morphology the same way, no gradient can separate them and the loss stalls on a valley floor — a real hazard of inverse problems, and one of the questions below.

18.2 Run it

Run it.

```
cd differentiable/03_recover_parameters
python run.py                # FD-check the gradient, then recover (chi,
                             kappa)
```

It generates a target from truth $(\chi, \kappa) = (2.5, 0.01)$, checks the loss gradient against finite

differences, then recovers the material from the wrong initial guess. Compare with `EXPECTED.md`.

Measured self-check (6-step rollout, L-BFGS-B on the adjoint gradient).

parameter	truth	recovered	error
χ (interaction)	2.5	2.50000	2.2×10^{-11}
κ (gradient pen.)	0.01	0.01000	1.8×10^{-11}

Field misfit L : 6.17×10^{-2} (initial guess) $\rightarrow 1.6 \times 10^{-20}$ (recovered), in 13 optimiser evaluations.

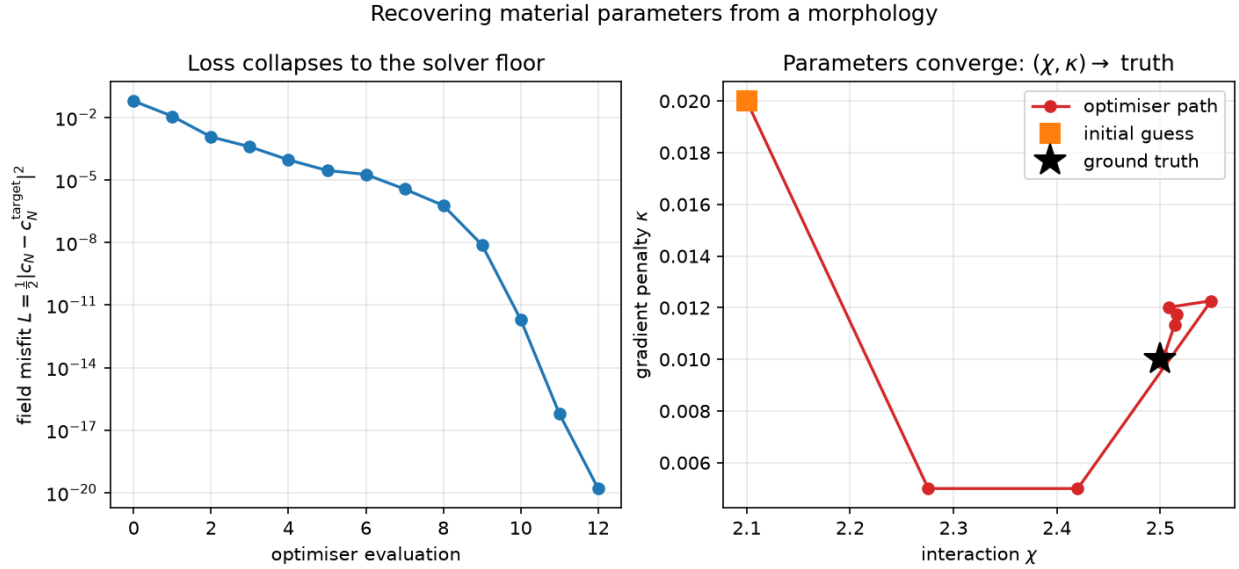


Figure 18.1: Recovering the material from the morphology. Left: the field misfit (18.1) collapses to the solver floor — the target was produced by the forward model, so a perfect fit exists and gradient descent finds it. Right: the optimiser’s path in (χ, κ) space, from the wrong initial guess (square) to the ground truth (star).

18.3 Code walk-through

The objective-and-gradient the optimiser calls is the whole method:

```
def loss_and_grad(theta, target):
    fwd, cN = march(dm, coords, chi=theta[0], kappa=theta[1], ...)
    r = cN - target; L = 0.5 * (r @ r) # eq. (D3)
    dJdx[-1][0::2] = r # dL/dc_N
    g = CHAdjoint(fwd).gradient(dJdx, ["B", "kappa"])
    return L, np.array([g["B"], g["kappa"]])

minimize(loss_and_grad, theta0, jac=True, method="L-BFGS-B", bounds=...)
```

scipy’s L-BFGS-B does the rest. Bounds keep the trial parameters in the interior region where the Flory–Huggins logs stay smooth — a reminder that inverse problems inherit the forward model’s

domain of validity.

18.4 Explore on your own

Q18.4.1 Fit *only* χ with κ held at the truth, then fit both. Does the joint problem converge as cleanly, or do the two parameters trade off early in the descent?

Q18.4.2 Corrupt the target with noise ($c^{\text{target}} \rightarrow c^{\text{target}} + \sigma \xi$). How does the recovered (χ, κ) degrade with σ , and does the loss floor rise to $\sim \sigma^2$ times the number of nodes?

Q18.4.3 Start the guess on the *wrong side* of the spinodal ($\chi_0 < 2$). Does the optimiser still find the demixing truth, or does a vanishing early gradient trap it in the mixed state?

Chapter 19

Process gradients

Key idea. Material parameters are what the blend *is*; *process* parameters are what we *do* to it. The headline process control is the temperature schedule $T(t)$ — the quench — applied while the film crystallises. The adjoint returns the gradient of a morphology objective with respect to the *whole schedule at once*, as a *time series* dJ/dT_n : one number per step, from a single reverse sweep. That is what makes designing a process (a long schedule) tractable.

19.1 Temperature enters everywhere

The physics is the coupled Cahn–Hilliard $\times$ Allen–Cahn crystallisation system (Physics P6–P7): a conserved composition ϕ and a non-conserved crystallinity ψ . Temperature enters each step’s residual through *two* channels:

$$\text{drive} = \Delta h \left(\frac{T}{T_m} - 1 \right) \quad (\text{Turnbull crystallisation}), \quad B(T) = B_0 + b_T (T - T_{\text{ref}}) \quad (\text{Flory interaction}). \quad (19.1)$$

So a change in T at step n ripples through both the crystallisation thermodynamics and the demixing thermodynamics of that step, and then forward through every later step via the coupled history. The adjoint sweep runs that dependency backward and returns dJ/dT_n for all n simultaneously.

One sweep, N gradients. A finite-difference schedule gradient costs $2N$ extra forward solves (perturb each T_n up and down); the adjoint costs *one* backward solve, independent of the schedule length N . For a schedule sampled every time step — hundreds of controls — only the adjoint is affordable. This is the same $O(1)$ -in- parameters advantage as D1, now paying off on a genuinely high- dimensional control.

19.2 Run it

Run it.

```
cd differentiable/04_process_gradients
python run.py                # dJ/dT_n time series, FD-verified per step
```

It marches a 4-step crystallisation under the schedule $T = [0.5, 0.6, 0.7, 0.55]$, forms an objective on the final ϕ, ψ , and returns the whole dJ/dT_n series, checking each entry against finite

differences. Compare with EXPECTED.md.

Measured self-check (schedule gradient of a 4-step BDF2 crystallisation, target $\langle\psi\rangle = 0.45$).

step n	T_n	dJ/dT_n (adjoint)
0	0.50	3.826×10^{-1}
1	0.60	2.998×10^{-1}
2	0.70	3.328×10^{-1}
3	0.55	2.822×10^{-1}

Worst adjoint-vs-FD agreement over the schedule: 6.8×10^{-10} (N gradients checked, not one).

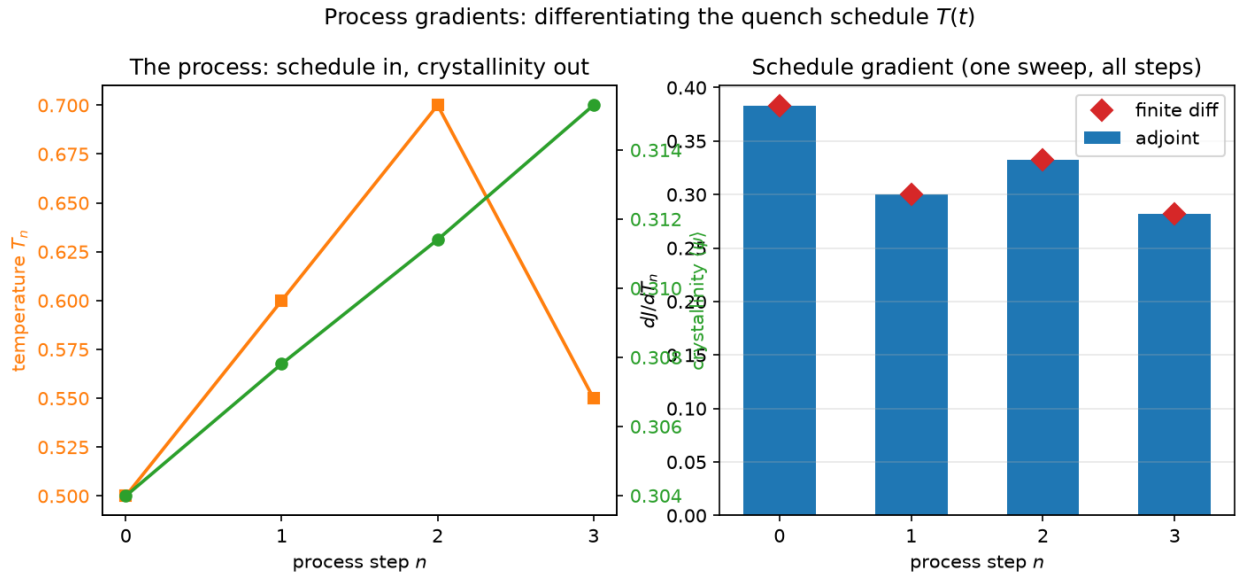


Figure 19.1: Differentiating the quench. Left: the imposed schedule T_n (orange) and the crystallinity $\langle\psi\rangle$ it produces (green) — the process, input and output. Right: the schedule gradient dJ/dT_n (adjoint bars, finite-difference diamonds), one value per step from a single reverse sweep. The bars say how much the temperature at each moment of the process matters to the final morphology.

19.3 Code walk-through

```
fwd = CACHForward(dm, FHEnergy(A, B), ..., dt=dt, order=2)
fwd.set_initial(phi0, psi0)
fwd.set_schedule(Tsched, bT=bT, Tref=Tref) # per-step temperature control
fwd.run(len(Tsched))
...
gT = CACHAdjoint(fwd).temperature_gradient(dJdx) # the whole time series
```

`set_schedule` makes each step read its temperature from `Tsched[n]`; `temperature_gradient` runs the reverse sweep and, at each step, contracts the adjoint state against $\partial R_n / \partial T$ (both the drive

and the b_T chi channel) — returning the length- N vector dJ/dT_n .

19.4 Explore on your own

Q19.4.1 Which step's temperature matters most, and why? Relate the ranking of $|dJ/dT_n|$ to when crystallisation is most active in the $\langle\psi\rangle$ trace.

Q19.4.2 Set $b_T = 0$ (decouple T from the Flory interaction). How much of each dJ/dT_n came from the demixing channel versus the crystallisation drive?

Q19.4.3 Lengthen the schedule to $N = 12$ and time the adjoint against a finite-difference gradient. Confirm the adjoint cost is flat while the FD cost grows like N — the crossover D1 anticipated, now on a real process control.

Chapter 20

Crystallinity sensitivity

Key idea. Device performance often hinges on *crystallinity* — how much of the film has ordered. This concept differentiates a crystallinity metric through the coupled Cahn–Hilliard $\times$ Allen–Cahn rollout with respect to the thermodynamic crystallisation parameters $(\Delta h, T_m, \Delta\sigma, \varepsilon^2, L)$ — all five from one adjoint sweep, each finite-difference verified. Now the sensitivities flow through the *non-conserved* crystallinity field ψ , not just the composition.

20.1 Differentiating the crystalline order

The crystallisation free energy (Physics P6) adds to the demixing energy a driven Allen–Cahn field ψ :

$$f = f_{\text{FH}}(\phi) + \phi[q(\psi)\Delta\sigma + p(\psi)\text{drive}], \quad \text{drive} = \Delta h\left(\frac{T}{T_m} - 1\right), \quad (20.1)$$

with $q(\psi) = \psi^2(1 - \psi)^2$ the double-well barrier (height $\Delta\sigma$) and $p(\psi) = \psi^2(3 - 2\psi)$ the monotone interpolation. The crystallinity relaxes by $\dot{\psi} = -L[\partial f/\partial\psi - \varepsilon^2\nabla^2\psi]$. Five parameters shape it: the latent heat Δh and melting point T_m set the *driving force*, the barrier $\Delta\sigma$ *opposes* ordering, the gradient penalty ε^2 sets *interface stiffness*, and L sets the *rate*. We take a crystallinity metric $J = \frac{1}{2}\|\psi_N - \psi^*\|^2$ and let the adjoint report how each moves it.

Reading the signs. At the tutorial setting the mean crystallinity $\langle\psi\rangle \approx 0.3122$ sits *below* the target $\psi^* = 0.4$. So any parameter that *raises* crystallinity lowers J : raising the latent heat Δh deepens the drive ($dJ/d\Delta h < 0$), while raising the barrier $\Delta\sigma$ resists ordering ($dJ/d\Delta\sigma > 0$). The gradient’s sign is a statement of physics, and it is exactly what finite differences confirm.

20.2 Run it

Run it.

```
cd differentiable/05_crystallinity_sensitivity
python run.py # dJ/d{dh, Tm, dsig, eps2, L}, FD-verified
```

It marches a 3-step crystallisation, forms $J = \frac{1}{2}\|\psi_N - 0.4\|^2$, and prints the five sensitivities with their finite-difference checks. Compare with `EXPECTED.md`.

Measured self-check (3-step crystallisation, $\langle\psi\rangle = 0.3122$, target $\psi^* = 0.4$).

parameter	adjoint dJ/dp	role
Δh (latent heat)	-8.325×10^{-2}	driving force
T_m (melting temp)	-1.388×10^{-2}	drive sign
$\Delta\sigma$ (barrier)	1.640×10^{-2}	opposes ordering
ε^2 (stiffness)	-1.673×10^{-1}	interface width
L (AC kinetics)	-7.191×10^{-2}	relaxation rate

Worst adjoint-vs-FD agreement: 9.9×10^{-10} .

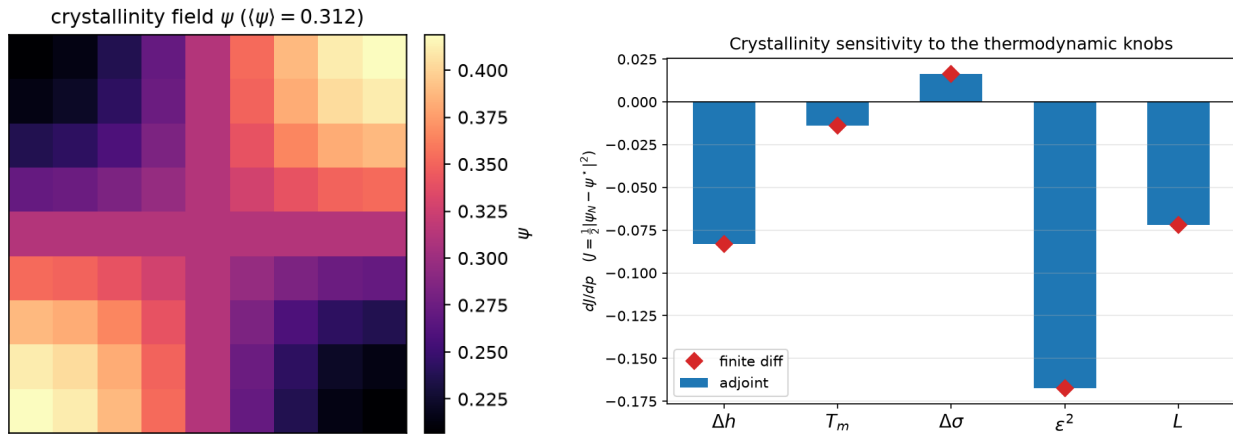


Figure 20.1: Left: the crystallinity field ψ whose scalar summary we differentiate. Right: the five crystallisation sensitivities (adjoint bars, finite-difference diamonds). Their relative sizes rank the knobs: the interface stiffness ε^2 and the latent heat Δh move the crystallinity most at this setting.

20.3 A frontier, named honestly

Noise is a different derivative. Nucleation is seeded by a *stochastic* term (Physics P8). The adjoint here differentiates the deterministic crystallisation *drift*. A sensitivity to the *noise amplitude* is a derivative of an *expectation* — a pathwise or score-function estimator, not a single-trajectory adjoint — and is a documented frontier. The tutorial differentiates the drift parameters the merged adjoint covers, and stops there rather than present a stochastic gradient it has not verified.

20.4 Explore on your own

Q20.4.1 Push T below and above T_m . The drive $\Delta h(T/T_m - 1)$ changes sign at $T = T_m$; does $dJ/d\Delta h$ flip with it, and does $\langle\psi\rangle$ melt back down above T_m ?

Q20.4.2 Rank the five sensitivities by magnitude and design the *cheapest* intervention: which single parameter, moved a little, changes the crystallinity most? Does the ranking survive a longer rollout?

Q20.4.3 Add the composition target back (differentiate $J = \frac{1}{2}(\|\phi_N - \phi^*\|^2 + \|\psi_N - \psi^*\|^2)$) and watch the demixing parameters M, κ, χ re-enter the gradient. Which parameters couple demixing and crystallisation?

Chapter 21

Learning the free energy from snapshots

Key idea. D3 recovered scalar *parameters* of a known free energy. Now recover the free-energy *functional* itself: given a time series of morphology snapshots, learn the bulk energy $f(c)$ that produced them. This is *learning the thermodynamics from the morphology* — the Wodo–Ganapathysubramanian programme — at tutorial scale, and it runs on exactly the same adjoint.

21.1 A learnable free energy

Parametrise the derivative of the bulk energy on a monomial basis,

$$f'(c) = \sum_i a_i c^{p_i}, \quad p = [3, 1], \quad \frac{\partial f'}{\partial a_i} = c^{p_i}, \quad (21.1)$$

so the coefficient sensitivity $\partial f'/\partial a_i$ is just the basis function — it drops straight into the adjoint's $\partial R_\mu/\partial p = -\int N(\partial f'/\partial p)$. The ground truth is the canonical double well $f'(c) = c^3 - c$, i.e. $a = [1, -1]$. We fit the a_i by minimising the misfit over a *trajectory* of snapshots,

$$L(a) = \sum_{n \in \mathcal{S}} \frac{1}{2} \|c_n(a) - c_n^{\text{data}}\|^2, \quad (21.2)$$

with $\mathcal{S}$ a chosen subset of steps; the gradient dL/da comes from one CHAdjoint sweep that seeds every snapshot step.

What a morphology cannot tell you. A constant term in $f'(c)$ merely shifts the chemical potential μ by a constant; the conserved c -dynamics ($c_t = \nabla \cdot M \nabla \mu$) are invariant to it. So a constant is *unidentifiable* from composition data, and the basis (21.1) excludes it on purpose. Knowing what your data *cannot* constrain is as important as the fit itself.

21.2 Run it

Run it.

```
cd differentiable/06_learn_free_energy
python run.py # learn  $f'(c)=c^3-c$  from snapshots
```

It generates a truth trajectory, FD-checks the trajectory-loss gradient, recovers the coefficients from clean data, and shows the noisy recovery sharpen as more snapshots are used. Compare with EXPECTED.md.

Measured self-check (learn $f'(c) = c^3 - c$ from 10 snapshots).

- **Clean recovery:** $a = [1.00000, -1.00000]$, $|a - \text{truth}|_\infty = 1.4 \times 10^{-11}$, loss 6.6×10^{-21} .
- **Noise beats down with data** ($\sigma = 0.01$, averaged over 12 seeds): recovery error 0.105 at $K=1$ snapshot $\rightarrow 7.0 \times 10^{-3}$ at $K=10$ — about $15\times$ better.

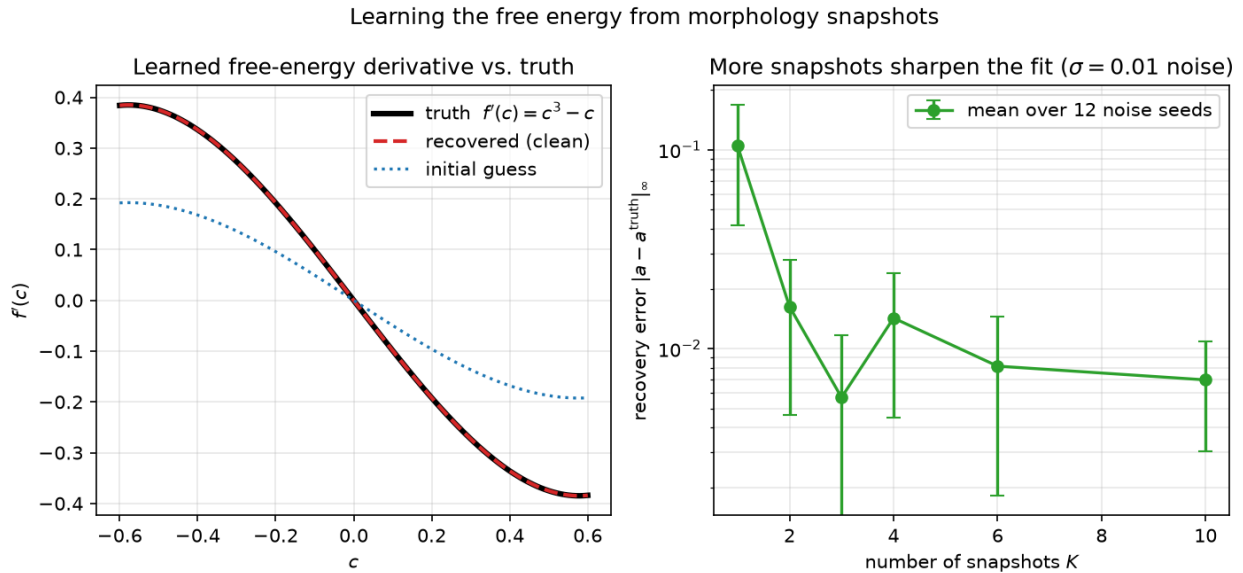


Figure 21.1: Learning the free energy from morphology. Left: the recovered $f'(c)$ (dashed) lies exactly on the truth $c^3 - c$ (black), starting from a wrong initial guess (dotted). Right: with noisy data the recovery error falls as more snapshots K are used — each snapshot is an independent constraint on the same coefficients, so a trajectory beats a single frame.

21.3 Code walk-through

```
f = CHForward(dm, PolyBasisEnergy([3,1], a), M=M, kappa=kappa, ...) #  $f'=a_0 c^3 + a_1 c$ 
f.set_initial(c0); f.run(nst)
for n in snaps: # trajectory misfit
    r = f.steps[n]["c"] - data[n]; L += 0.5 * (r @ r); dJ[n][0::2] = r
g = CHAdjoint(f).gradient(dJ, ["a0", "a1"]) #  $dL/da_i$  via  $df'/da_i = c^{\{p_i\}}$ 
minimize(loss_and_grad, a0, jac=True, method="L-BFGS-B")
```

The only new ingredient over D3 is the *basis* energy: because $\partial f'/\partial a_i$ is analytic, the same adjoint that returned material-parameter gradients now returns *functional* gradients. An MLP free energy would slot into the identical loop (the autograd twin already differentiates an arbitrary f') — the documented next step.

21.4 Explore on your own

Q21.4.1 Add a spurious basis term (e.g. c^5) the truth does not use. Does the fit drive its coefficient to zero, and how much data does that take?

Q21.4.2 Try to also recover a constant term $a c^0$. Watch it stay unidentified no matter how much data you add, and connect that to the conservation argument above.

Q21.4.3 Increase the noise σ and re-plot error-vs- K . Does the floor scale like σ , and does the number of snapshots needed for a given accuracy grow as you would expect from averaging?

Chapter 22

Capstone: inverse-designing a process

Key idea. The capstone closes the loop. Everything before this *computed* a gradient; here we *use* it to design. This is PDE-constrained optimisation: find a temperature schedule $T(t)$ that drives the crystallisation to a *target crystalline fraction*, by minimising an objective over the whole schedule with the D4 schedule gradient. The simulator, run backward, returns the recipe.

22.1 Design as optimisation

Specify what you want — here a target crystalline fraction ϕ^* — as an objective over the process control $T(t)$:

$$\min_{T \in [T_{\min}, T_{\max}]^N} J(T) = \frac{1}{2} (\langle \psi_N \rangle(T) - \phi^*)^2, \quad (22.1)$$

where $\langle \psi_N \rangle$ is the mean crystallinity the schedule produces. Each optimiser step needs J and its gradient dJ/dT_n ; the adjoint returns the whole time-series gradient (D4) from one reverse sweep, and a bounded quasi-Newton method (L-BFGS-B) walks the schedule downhill. The bounds keep the temperatures physical.

A reachable target, so we can check. The target is generated by a ground-truth cold ramp, so a schedule that hits it provably exists — which lets the tutorial *verify* that the designed process reaches the specification, not merely that the optimiser stopped. Point the same machinery at a real specification and it returns a candidate recipe.

22.2 Run it

Run it.

```
cd differentiable/07_inverse_design_capstone
python run.py # design T(t) to hit a target crystallinity
```

It sets a target from a ground-truth ramp, FD-checks the design gradient, then optimises the schedule from a flat guess and confirms the achieved fraction reaches the target. Compare with EXPECTED.md.

Measured self-check (design a 6-step schedule to a target crystalline fraction).

quantity	value
target crystalline fraction	0.23877
flat guess $T = 1$ gives	0.21909
designed schedule achieves	0.23877
achieved – target	4.2×10^{-15}
objective J : initial → final	$1.94 \times 10^{-4} \rightarrow 8.7 \times 10^{-30}$
optimiser evaluations	7

The designed schedule is interior (no step pinned at the bounds).

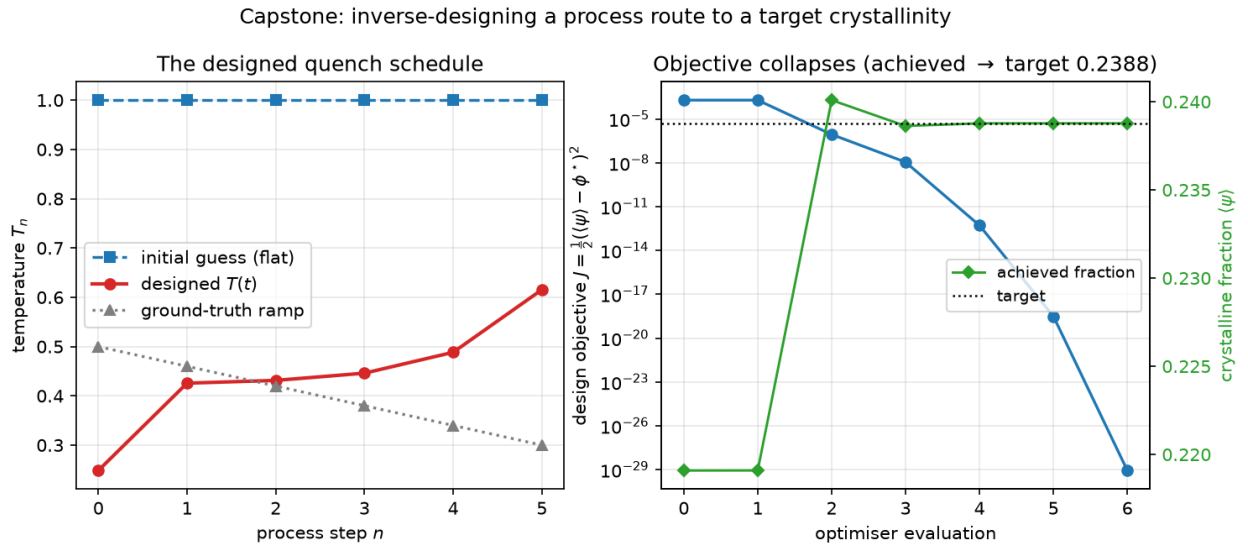


Figure 22.1: Inverse-designing the process. Left: the schedules — the flat initial guess, the designed $T(t)$, and the ground-truth ramp that set the target. Right: the design objective (22.1) collapses as the achieved crystalline fraction (green) climbs to the target (dotted). A handful of adjoint gradients suffice.

22.3 Code walk-through

The objective-and-gradient is the D4 schedule gradient wrapped in an optimiser:

```
def obj_and_grad(T, target):
    f = march(dm, coords, T, dt)
    m = mean(f.steps[-1]["x"][2::3])
    J = 0.5 * (m - target)**2
    dJ[-1][2::3] = (m - target) / nn
    return J, CACHAdjoint(f).temperature_gradient(dJ)

minimize(obj_and_grad, T0, jac=True, method="L-BFGS-B", bounds=[(0.2, 2.0)]*NS)
```

That is the whole capstone: a forward crystallisation, a reverse sweep for the schedule gradient, and `scipy` to descend. Every earlier concept — the verified gradient (D1), the sensitivities (D2, D4, D5), the inverse fits (D3, D6) — was a rehearsal for this.

22.4 Explore on your own

Q22.4.1 Many schedules hit the *same* scalar fraction. Add a second term to (22.1) — e.g. a target demixing amplitude or domain size — and watch the designed schedule become more constrained. Does it move toward the ground-truth ramp?

Q22.4.2 Tighten the bounds until the target becomes *unreachable*. How does the optimiser report failure, and what does the residual objective tell you about the closest achievable crystallinity?

Q22.4.3 Add a smoothness penalty $\lambda \sum_n (T_{n+1} - T_n)^2$ to (22.1) (real quenches cannot jump). How does the designed schedule change as λ grows, and what does it cost in achieved accuracy?

Q22.4.4 Swap the objective for a target on the *composition* field ϕ instead of the crystallinity ψ . Which process controls matter now, and does the coupled system still admit a schedule that hits it?